

1. Классификация ЭВМ. Основные принципы организации различных классов ЭВМ. (Моё мнение - убанский вопрос, я же блять 5 классификаций приведу в пример)

Электронная вычислительная машина, компьютер - комплекс технических средств, предназначенных для автоматической обработки информации в процессе решения вычислительных и информационных задач.

Чтобы **не запутаться** во всём этом многообразии принято классифицировать их по ряду признаков. В частности:

- по поколениям;
- по областям применения;
- по назначению;
- по вычислительной мощности;
- по роли в вычислительной сети;
- по функциональным возможностям;
- по характеру решаемых задач;
- по принципу действия;
- по способам организации вычислительного процесса;
- по конструктивным особенностям.

Классификация ЭВМ по принципу действия

По принципу действия **вычислительные машины** делятся на три больших класса:

- аналоговые (АВМ),
- цифровые (ЦВМ),
- гибридные (ГВМ).

Цифровые вычислительные машины (ЦВМ) - вычислительные машины дискретного действия, работают с информацией, представленной в дискретной, а точнее, в цифровой форме.

Аналоговые вычислительные машины (АВМ) - вычислительные машины непрерывного действия, работают с информацией, представленной в непрерывной (аналоговой) форме, т.е. в виде непрерывного ряда значений какой-либо физической величины (чаще всего электрического напряжения).

Гибридные вычислительные машины (ГВМ) — вычислительные машины комбинированного действия, работают с информацией, представленной и в цифровой, и в аналоговой форме; они совмещают в себе достоинства АВМ и ЦВМ.

Критерием деления вычислительных машин на эти три класса является форма представления информации, с которой они работают (рис. 6).

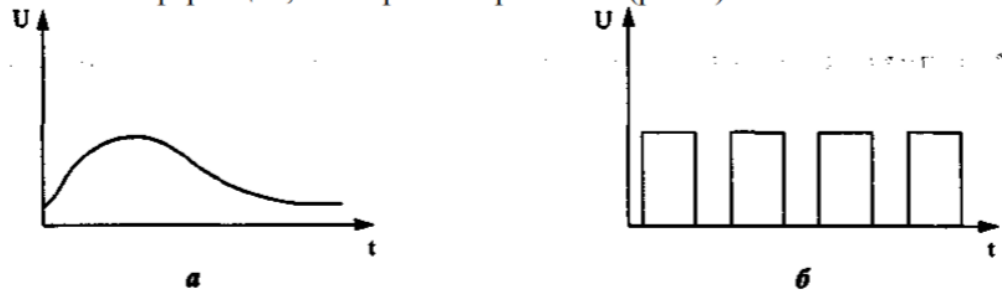


Рис. 6. Две формы представления информации в машинах:

- а - аналоговая;
- б - цифровая импульсная.

АВМ быстрее, но менее точные, на них удобно решать задачи, содержащие дифференциальные уравнения без сложной логики.

ГВМ целесообразно использовать для решения задач управления сложными быстродействующими техническими комплексами.

Наиболее широкое применение получили ЦВМ с электрическим представлением дискретной информации - электронные цифровые вычислительные машины, обычно называемые просто электронными вычислительными машинами (ЭВМ), без упоминания об их цифровом характере.

Классификация ЭВМ по этапам создания.

- 1-е поколение, 50-е гг.: ЭВМ на электронных вакуумных лампах;
- 2-е поколение, 60-е гг.: ЭВМ на дискретных полупроводниковых приборах (транзисторах);
- 3-е поколение, 70-е гг.: ЭВМ на полупроводниковых интегральных схемах с малой и средней степенью интеграции (сотни - тысячи транзисторов в одном корпусе);
- 4-е поколение, 80-е гг.: ЭВМ на больших и сверхбольших интегральных схемах — микропроцессорах (десятки тысяч - миллионы транзисторов в одном кристалле);
- 5-е поколение, 90-е гг.: ЭВМ с многими десятками параллельно работающих микропроцессоров, позволяющих строить эффективные системы обработки знаний; ЭВМ на сверхсложных микропроцессорах с параллельно-векторной структурой, одновременно выполняющих десятки последовательных команд программы;
- 6-е и последующие поколения: оптоэлектронные ЭВМ с массовым параллелизмом и нейронной структурой - с распределенной сетью большого числа (десятки тысяч) несложных микропроцессоров, моделирующих архитектуру нейронных биологических систем.

Интегральная схема - электронная схема специального назначения, выполненная в виде единого полупроводникового кристалла, объединяющего большое число диодов и транзисторов.

Классификация ЭВМ по назначению

По назначению ЭВМ можно разделить на три группы:

универсальные (общего назначения), проблемно-ориентированные, специализированные.

Универсальные ЭВМ предназначены для решения самых различных инженернотехнических задач: экономических, математических, информационных и других задач, отличающихся сложностью алгоритмов и большим объемом обрабатываемых данных.

Характерными чертами универсальных ЭВМ являются:

- высокая производительность;
- разнообразие форм обрабатываемых данных: двоичных, десятичных, символьных, при большом диапазоне их изменения и высокой точности их представления;
- обширная номенклатура выполняемых операций, как арифметических, логических, так и специальных;
- большая емкость оперативной памяти;
- развитая организация системы ввода-вывода информации, обеспечивающая подключение разнообразных видов внешних устройств.

Проблемно-ориентированные ЭВМ служат

- для решения более узкого круга задач, связанных, как правило, с управлением технологическими объектами;
- регистрацией, накоплением и обработкой относительно небольших объемов данных; выполнением расчетов по относительно несложным алгоритмам;
- обладают ограниченными по сравнению с универсальными ЭВМ аппаратными и программными ресурсами.

К проблемно-ориентированным ЭВМ можно отнести, в частности, всевозможные управляющие вычислительные комплексы.

Специализированные ЭВМ используются для решения узкого круга задач или реализации строго определенной группы функций. Такая узкая ориентация ЭВМ позволяет четко специализировать их структуру, существенно снизить их сложность и стоимость при сохранении высокой производительности и надежности их работы.

К специализированным ЭВМ можно отнести, например, программируемые микропроцессоры специального назначения; адаптеры и контроллеры, выполняющие логические функции управления отдельными несложными техническими устройствами, агрегатами и процессами; устройства согласования и сопряжения работы узлов вычислительных систем.

Классификация ЭВМ по размерам и функциональным возможностям

По размерам и функциональным возможностям ЭВМ можно разделить на:

- сверхбольшие (суперЭВМ),
- большие,
- малые,
- сверхмалые (микроЭВМ).

Сравнительные параметры классов современных ЭВМ:

Параметр	СуперЭВМ	Большие	Малые ЭВМ	МикроЭВМ
Производительность, MIPS	1000-100000	10-1000	1-100	1-100
Емкость ОП, Мбайт	2000-10000	64-10000	4-512	4-256
Емкость ВЗУ, Гбайт	500-5000	50-1000	2-100	0,5 - 10
Разрядность, бит	64-128	32-64	16-64	16-64

Супер ЭВМ — сверхпроизводительные ЭВМ, предназначенные для решения особо сложных задач в областях науки, техники и управления, например, таких как прогнозирование метеообстановки, ядерная энергетика, оборона и т.д.

Самые мощные ЭВМ используют тысячи процессоров, потребляют мегаватты энергии и занимают целые этажи зданий. Такие машины обладают колоссальным быстродействием в миллиарды операций в секунду, основанном на выполнении параллельных вычислений и использовании многоуровневой иерархической структуры запоминающих устройств, требуют для своего размещения специальных помещений и крайне сложны в эксплуатации. Стоимость отдельной ЭВМ такого класса достигает десятки миллионов долларов.

Большие ЭВМ — ЭВМ, имеющие высокую производительность, большой объем основной и внешней памяти. Они обладают способностью параллельной обработки данных и обеспечивают как пакетный, так и интерактивный (диалоговый) режимы работы.

Большие ЭВМ предназначены для выполнения работ, связанных с обработкой и хранением больших объемов информации, проведением сложных расчетов и исследований в ходе решения вычислительных и информационно-логических задач.

Средние ЭВМ — ЭВМ производительностью до нескольких миллионов операций в секунду, с емкостью оперативной памяти в несколько десятков мегабайт и разрядностью от 32.

Вычислительные машины этого класса обладают несколько меньшими возможностями, чем большие ЭВМ, но зато им присуща и более низкая стоимость. Они предназначены для использования всюду, где приходится постоянно обрабатывать достаточно большие объемы информации с приемлемыми временными затратами.

Малые ЭВМ (мини-ЭВМ) — надежные, недорогие и удобные в эксплуатации компьютеры, обладающие несколько низкими параметрами по сравнению с большими ЭВМ.

Основными особенностями являются:

- широкий диапазон производительности в конкретных условиях применения,
- аппаратная реализация большинства системных функций ввода-вывода информации,
- простая реализация микропроцессорных и многомашинных систем,
- высокая скорость обработки прерываний,
- возможность работы с форматами данных различной длины.

Микро-ЭВМ – это компьютеры, в которых центральный процессор выполнен в виде одной микросхемы (микропроцессора). Современные модели микро-ЭВМ могут содержать несколько микропроцессоров. Можно выделить следующие группы в классе микро-ЭВМ:



Рис. 9. Классификация микроЭВМ

Многопользовательские микроЭВМ - это мощные микроЭВМ, оборудованные несколькими видеотерминалами и функционирующие в режиме разделения времени, что позволяет эффективно работать на них сразу несколькими пользователями.

Персональные компьютеры (ПК) - однопользовательские микроЭВМ удовлетворяющие требованиям **общедоступности и универсальности** применения.

Рабочие станции (work station) представляют собой однопользовательские мощные микроЭВМ, специализированные для выполнения **определенного вида работ** (графических, инженерных, издательских и др.).

Серверы (server) - многопользовательские мощные микроЭВМ в вычислительных сетях, выделенные для обработки запросов от всех станций сети.

Классификация по способу организации вычислительного процесса

- многопроцессорные
- однопроцессорные
- параллельные
- последовательные

В многопроцессорных ЭВМ архитектура предусматривает использование большого числа процессоров, чем обеспечивается существенное повышение ее вычислительной мощности и, в частности, возможность обработки значительных объемов информации.

В однопроцессорных ЭВМ быстродействующая память подключена к единственному центральному процессору, который выбирает команды из памяти, декодирует каждую из них, выбирает данные (в соответствии с предписанием в команде), хранящиеся в указанных ячейках памяти, выполняет указанные операции и запоминает результаты в предписанных ячейках. Кроме того, к системе должны быть подключены устройства ввода и вывода.

Параллельная организация вычислительного процесса предполагает разбиение решаемой задачи на какие-либо части и дальнейшее одновременное их выполнение.

Последовательная организация вычислительного процесса предполагает последовательное выполнение программы, т.е. команда за командой согласно логике программы.

2. Системы счисления, используемые в ЭВМ. Перевод чисел из одной системы в другую.

Системой счисления называется способ представления чисел посредством алфавита символов

В ЭВМ используются позиционные системы счисления: двоичная, восьмеричная, шестнадцатеричная

Правила перевода чисел из двоичной (шестнадцатеричной) системы счисления в десятичную систему счисления:

Для перевода числа из двоичной (шестнадцатеричной) системы счисления в десятичную систему счисления число представляется в виде полинома, где все цифры выражаются в десятичной системе счисления и действия выполняются в десятичной системе счисления.

Пример: перевести число 11000, выраженное в двоичной системе счисления, в десятичную систему счисления:

$$(11000)_2 = (?)_{10}$$
$$43210$$

$$(11000)_2 = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 16 + 8 = (24)_{10}$$

Правила перевода чисел из десятичной системы счисления в двоичную (шестнадцатеричную) систему счисления.

1. Правила перевода целых чисел.

Чтобы перевести целое число из десятичной системы счисления в двоичную (шестнадцатеричную) систему счисления, необходимо исходное число разделить на основание новой системы счисления

2. Правила перевода дробной части числа.

Чтобы перевести правильную дробь из десятичной системы счисления в двоичную (шестнадцатеричную) систему счисления, необходимо исходную дробь умножить на основание новой системы счисления.

Правила перевода из двоичной системы счисления в шестнадцатеричную систему счисления.

Разбить число на тетрады. Каждую тетраду необходимо заменить шестнадцатеричной цифрой

Правила перевода из шестнадцатеричной системы счисления в двоичную систему счисления.

Каждую шестнадцатеричную цифру заменить эквивалентной тетрадой в двоичной системе счисления

3. Формы представления чисел в ЭВМ. Точность, диапазон представления чисел с плавающей точкой.

Числа с фиксированной точкой — это двоичные числа, у которых ограничен размер их целой и дробной части.

Формат представления чисел с фиксированной точкой

Рассмотрим формат представления чисел с фиксированной запятой (рис. 1).

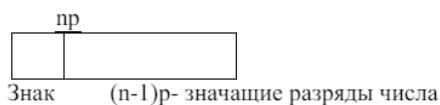


Рис. 1. Формат представления чисел с фиксированной запятой

Точка может фиксироваться либо перед старшим разрядом числа, либо после младшего. Если точка фиксируется перед старшим разрядом числа, то все числа по модулю меньше единицы. Если после младшего, то все числа по модулю больше единицы.

В современных ЭВМ для представления целых чисел используется формат чисел с фиксированной точкой, причем точка фиксируется после младшего разряда.

Отрицательные числа с фиксированной точкой хранятся в памяти в обратном или в дополнительном коде.

Формат чисел с плавающей точкой

Число X с плавающей точкой можно представить по формуле:

$$X = \pm M_x \cdot q^{\pm p_x}$$

Где: M_x – мантисса числа,

q – основание системы счисления,

p – порядок.

Мантисса – это нормализованная правильная дробь.

Формат чисел с плавающей точкой представлен на рис. 2.

Тогда:

$$P_{см} = P_x + 2^L - \text{смещенный порядок},$$

где L – количество разрядов, отводимое под порядок.

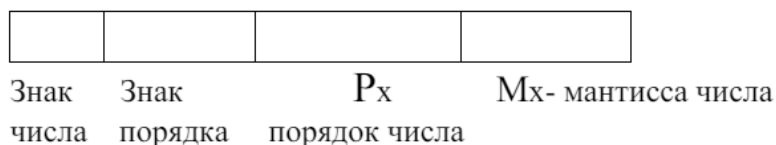


Рис. 2. Формат чисел с плавающей точкой

В современных ЭВМ часто для представления мантиссы используется либо двоичная, либо шестнадцатеричная системы счисления. Ниже для наглядности рассматриваются примеры использования шестнадцатеричной системы счисления для представления мантиссы числа.

4. Представление двоичных чисел в прямом, обратном и дополнительном кодах. Сложение в дополнительном и обратном кодах.

Прямым кодом целого числа x является число, образованное по формуле:

$$X = \begin{cases} 0X_{n-2} \dots X_1X_0 & , \quad X \geq 0 \\ 1X_{n-2} \dots X_1X_0 & , \quad X < 0 \end{cases}$$

Обратным кодом целого числа x является число, образованное по формуле:

$$X = \begin{cases} 0X_{n-2} \dots X_1X_0 & , \quad X \geq 0 \\ 1\bar{X}_{n-2} \dots \bar{X}_1\bar{X}_0 & , \quad X < 0 \end{cases}$$

Для 2 с/с обратный код отрицательного числа получается путём инвертирования значащих разрядов, т.е. нули заменяются единицами и наоборот - единицы нулями, а в знаковый разряд ставится 1.

Для положительных чисел прямой и обратный код совпадают.

Суммирование с обратным кодом:

Отрицательные числа представляются в обратном коде, положительные – в прямом и производится поразрядное суммирование, включая знаковый разряд.

Если возникает единица переноса из знакового разряда, то она суммируется с младшим разрядом (циклический перенос).

Если знаковый разряд суммы равняется "0", то это означает, что результат положительный и представлен в прямом коде.

Если в знаковом разряде суммы "1", то это означает, что результат отрицательный и представлен в обратном коде.

Дополнительным кодом целого числа x является число, образованное по формуле:

$$X = \begin{cases} 0X_{n-2} \dots X_1X_0 & , \quad X \geq 0 \\ 1\bar{X}_{n-2} \dots \bar{X}_1\bar{X}_0 + 1 & , \quad X < 0 \end{cases}$$

Суммирование с дополнительным кодом:

При возникновении единицы переноса из знакового разряда эта единица отбрасывается, в отличие от обратного кода.

Если в знаковом разряде суммы единица, то это означает, что результат отрицательный и представлен в дополнительном коде.

Что будет, если перевести 0 или 1 в обратный и доп код

Еще одним достоинством дополнительного кода является то, что 0, в отличие от прямого и обратного кодов, представляется одним кодом.

Десятичное число	Прямой код	Обратный код	Дополнительный код
0	1000 0000	1111 0000	0000
1	0001	0001	0001

5. Модифицированное кодирование. Определение переполнения разрядной сетки ЭВМ.

Переполнением разрядной сетки называется ситуация, когда при суммировании кодов чисел **результат требует на один разряд больше**, чем исходные операнды и не умещается в разрядной сетке ЭВМ

1. По наличию и отсутствию переноса в знаковый и из знакового разряда.

Переполнение возникает в том случае, если существует **только лишь один перенос**, либо в знаковый разряд, либо из знакового разряда.

2. Модифицированное кодирование.

Модифицированным кодом называется код, в котором **под знак числа отводится два или более разрядов**.

При использовании двух разрядов для представления знака числа комбинация знаковых разрядов 00 соответствует положительным числам, комбинация 11 – отрицательным числам, **комбинации 01 и 10 соответствуют переполнению разрядной сетки**.

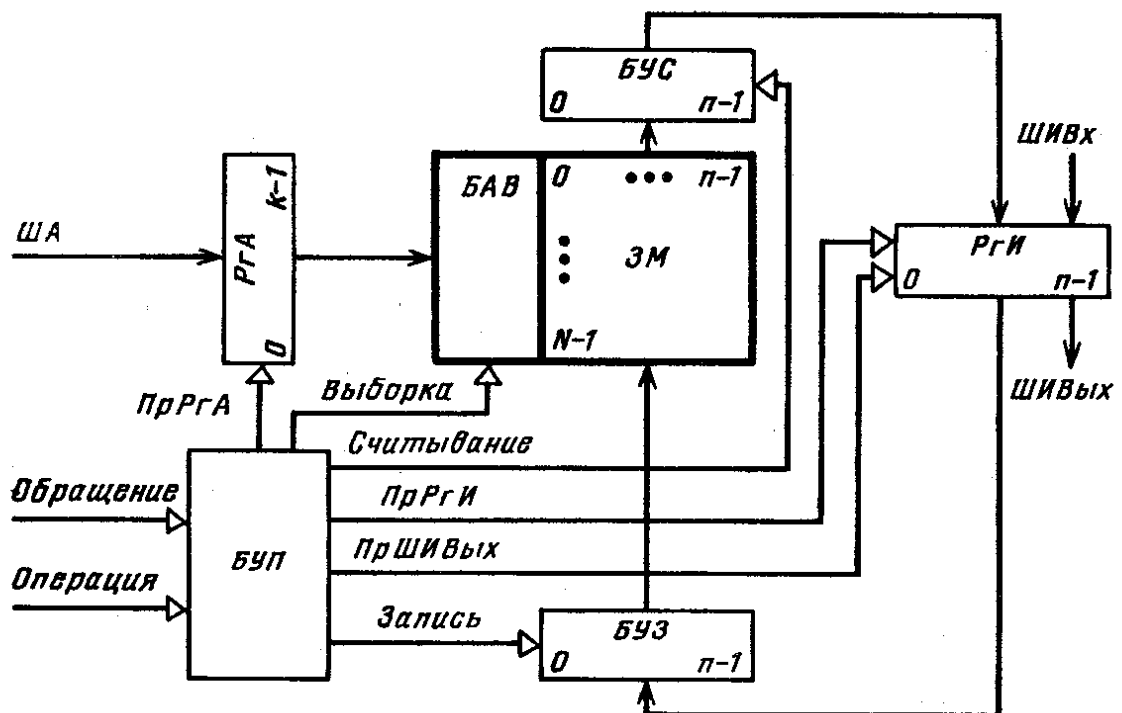
6. Структура адресных ЗУ, стековых ЗУ, ассоциативных ЗУ.

(взято с <https://studfile.net/preview/5179826/page:2/>)

Адресная память. В памяти с адресной организацией размещение и поиск информации в ЗМ основаны на использовании адреса хранения слова (числа, команды и т. п.), адресом служит номер ячейки ЗМ, в которой это слово размещается.

При записи (или считывании) слова в ЗМ инициирующая эту операцию команда должна указывать адрес (номер ячейки), по которому производится запись (считывание).

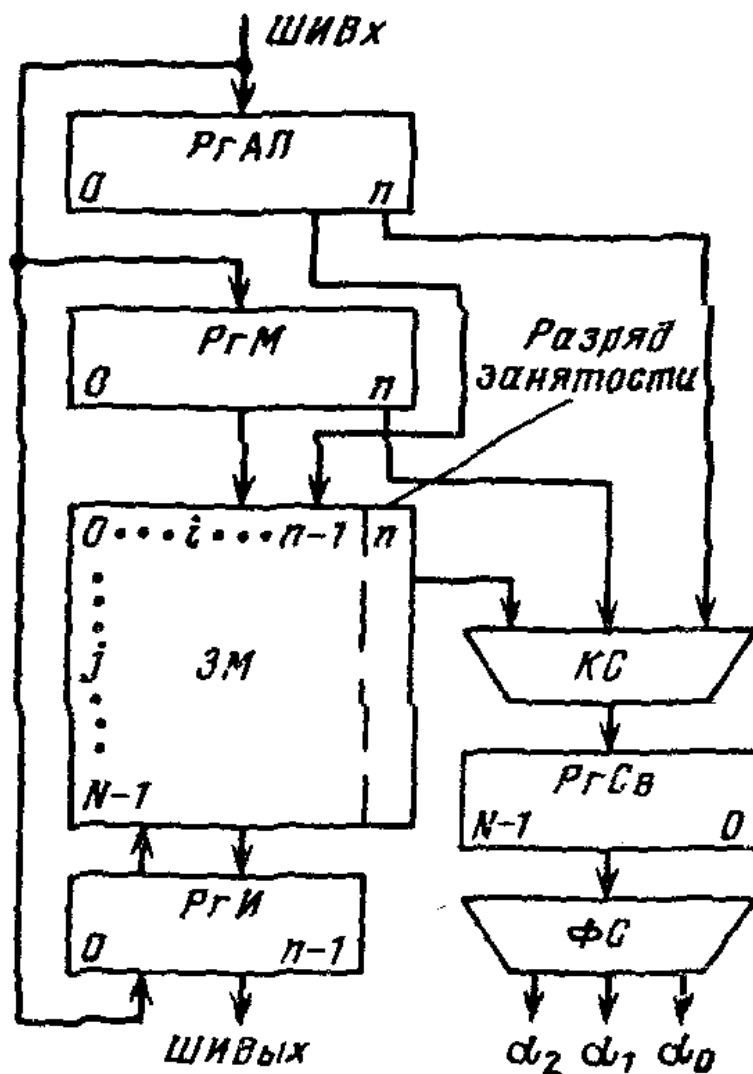
Типичная структура адресной памяти, показанная на рис. 4.2, содержит запоминающий массив из N n -разрядных ячеек и его аппаратное обрамление, включающее регистр адреса $РгА$, имеющий k ($k > \log_2 N$) разрядов, информационный регистр $РгИ$, блок адресной выборки $БАВ$, блок усилителей считывания $БУС$, блок разрядных усилителей-формирователей сигналов записи $БУЗ$ и блок управления памятью $БУП$.



Ассоциативная память. В памяти этого типа поиск нужной информации производится не по адресу, а по ее содержанию (по ассоциативному признаку). При этом поиск по ассоциативному признаку (или последовательно по отдельным разрядам этого признака) происходит параллельно во времени для всех ячеек запоминающего массива. Во многих случаях ассоциативный поиск позволяет существенно упростить и ускорить обработку данных. Это достигается за счет того, что в памяти этого типа операция считывания информации совмещена с выполнением ряда логических операций.

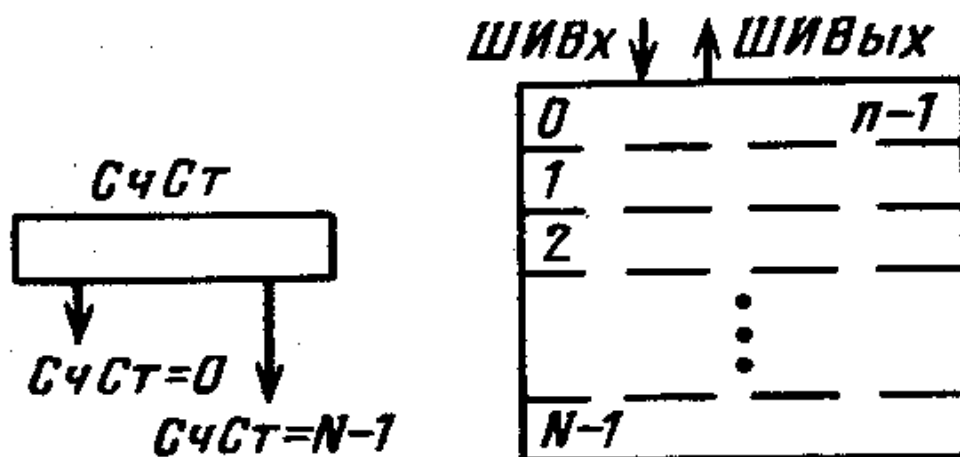
Типичная структура ассоциативной памяти представлена на рис. 4.3. Запоминающий массив содержит N ($n + 1$)-разрядных ячеек. Для указания занятости ячейки используется служебный n -й разряд (0 — ячейка свободна, 1 — в ячейке записано слово).

По входной информационной шине ШИВх в регистр ассоциативного признака $РгАП$ в разряды 0—и—1 поступает n -разрядный ассоциативный запрос, а в регистр маски $РгМ$ — код маски поиска, при этом n -й разряд $РгМ$ устанавливается в 0. Ассоциативный поиск производится лишь для совокупности разрядов $РгАП$, которым 'соответствуют 1 в $РгМ$ (незамаскированные разряды $РгАП$). Для слов, в которых цифры в разрядах совпали с незамаскированными разрядами $РгАП$, комбинационная схема $КС$ устанавливает 1 в соответствующие разряды регистра совпадения $РгСв$ и 0 в остальные разряды.



Стековая память, так же как и ассоциативная, является безадресной. В *стековой памяти* (рис. 4.4) ячейки образуют одномерный массив, в котором соседние ячейки связаны друг с другом разрядными цепями передачи слов. Запись нового слова производится в верхнюю ячейку (ячейку 0), при этом все ранее записанные слова (включая слово, находившееся в ячейке 0), сдвигаются вниз, в соседние ячейки с большими на 1 номерами. Считывание возможно только из верхней (нулевой) ячейки памяти, при этом, если производится считывание с удалением, все остальные слова в памяти сдвигаются вверх, в соседние ячейки с большими номерами. В этой памяти порядок считывания слов соответствует правилу: *последним поступил — первым обслуживается*. В ряде устройств рассматриваемого типа предусматривается также операция простого считывания слова из нулевой ячейки (без его удаления и сдвига слова в памяти). Иногда стековая память снабжается счетчиком стека *СчСт*, показывающим количество занесенных в память слов. Сигнал $СчСт = 0$ соответствует пустому, стеку, $СчСт = N - 1$ — заполненному стеку.

Часто стековую память организуют, используя адресную память. Широкое применение стековая память находит при обработке вложенных структур данных.



Мишань, выздоравливай (Спасибо)

7. Иерархическая организация многоуровневой памяти. Характеристики внутренней и внешней памяти ЭВМ.

Обобщённая модель структуры, отражающая многообразие ЗУ и их взаимодействие, представлена на **рис. 36**. Все запоминающие устройства обладают различным быстродействием и емкостью.

Чем **выше** уровень иерархии, тем **выше** быстродействие соответствующей памяти, но **меньше** её емкость.

- К самому высокому уровню - **сверхоперативному** - относятся **регистры управляющих и операционных блоков процессора**, **сверхоперативная память**, **управляющая память**, **буферная память (кэш-память)**.
- На **втором оперативном** уровне находится **оперативная память (ОП)**, служащая для хранения программ и данных, с которыми работает ЭВМ.
- На **следующем** более низком внешнем уровне размещается **внешняя память**.

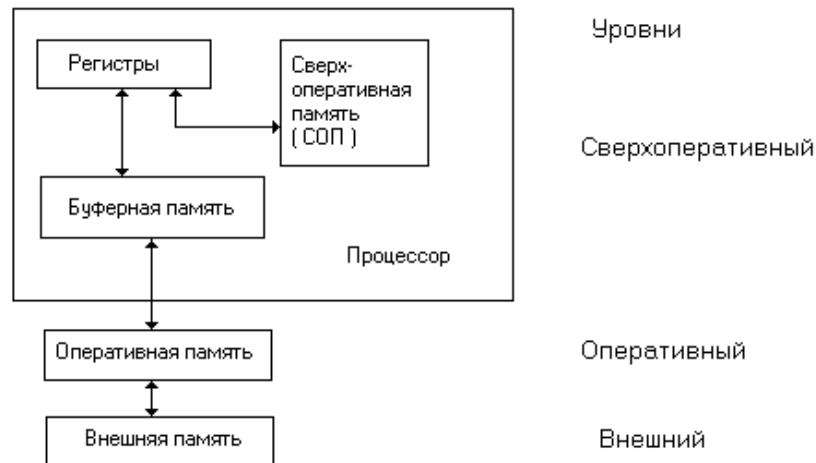


Рис. 36. Иерархическая структура памяти

1. **Местная память** (регистровая память процессора) входит в состав ЦП (регистры управляющих и операционных блоков процессора) и предназначена для **временного хранения информации**. Она имеет **малую ёмкость** и **наибольшее быстродействие**.

Такая память построена на базе **регистров общего назначения**, которые конструктивно совмещены с процессором ЭВМ. Этот тип ЗУ используется для **хранения управляющих и служебных кодов**, а также **информации**, к которой наиболее часто обращается процессор **при выполнении программы**.

Иногда в архитектуре ЭВМ регистровая память организуется в виде **сверхоперативного ЗУ** с прямой адресацией. Такая память служит для хранения операндов, данных и служебной информации, необходимой процессору.

2. **Управляющая память** предназначена для **хранения управляющих микропрограмм процессора** и выполняется в виде постоянного ЗУ (**ПЗУ**) или программируемого постоянного ЗУ (**ППЗУ**). В системах с микропрограммным способом обработки информации УП применяется для хранения однажды записанных микропрограмм, управляющих программ, констант.

3. В качестве **буферной памяти** рассматривается **кэш-память**, которая размещается между основной (оперативной) памятью и процессором. Основное назначение кэш-памяти - **кратковременное хранение** и **выдача активной информации процессору**, что сокращает число обращений к основной памяти, скорость работы которой меньше, чем кэш-памяти. Кэш-память не является программно доступной.

В современных ЭВМ различают кэш первого и второго уровней.

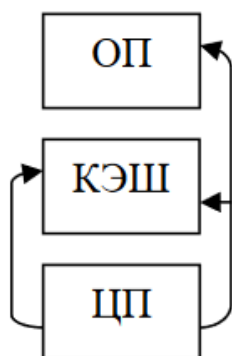
Кэш **первого уровня** интегрирована с блоком предварительной выборки команд и данных ЦП и служит, как правило, для хранения наиболее часто используемых команд.

Кэш **второго уровня** служит буфером между ОП и процессором. В некоторых ЭВМ существует кэш-память отдельно для команд и отдельно для данных.

4. **Оперативная память (ОЗУ)** служит для хранения информации, непосредственно **участвующей в вычислительном процессе**. Из ОЗУ в процессор поступают коды и операнды, над которыми производятся предусмотренные программой операции, из процессора в ОЗУ направляются для хранения промежуточные и конечные результаты обработки информации.

5. **Внешняя память (ВнП)** используется для **хранения больших массивов информации** в течении **продолжительного периода времени**. Обычно ВнП не имеет непосредственной связи с процессором. В качестве носителя используются магнитные диски (гибкие и жёсткие), лазерные диски и др.

8. Организация Кэш-памяти. Основные способы отображения ОП на БП.



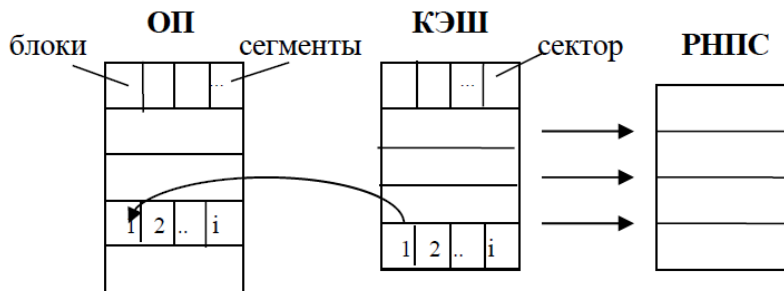
КЭШ невидима для программиста.

Когда ЦП обращается к ОП по чтению или записи информации (при чтении это обращение за командами, за данными), то аппаратно просматривается, находится ли запрашиваемая информация в КЭШ.

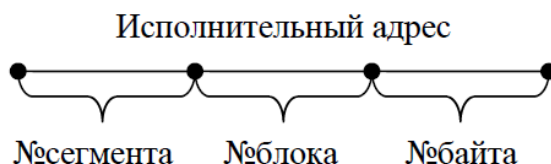
- Если информация **находится в КЭШ**, то обращение к ОП **не требуется**.
- Если запрашиваемая информация **в КЭШ отсутствует**, то **происходит** обращение к ОП. При этом часть информации из КЭШ удаляется и на ее место записывается информация из ОП.

Различают буфер прямого соответствия, секторный, группо-ассоциативный, ассоциативный способы организации КЭШ памяти.

9. Секторный способ отображения ОП на БП.



- 1) Объем сегмента равен объему сектора. Любой сегмент может храниться в любом секторе. Поскольку КЭШ меньше емкости ОП, количество секторов будет меньше количества сегментов.
- 2) Сегменты и сектора делятся на блоки равной величины. Число блоков в сегменте и секторе совпадают.
- 3) Каждому сектору в КЭШ соответствует регистр номера присутствующего сегмента (РНПС). РНПС хранит номер сегмента, находящегося в данный момент в данном секторе. i -ый блок j -ого сегмента может находиться только на месте i -ого блока k -ого сектора.
- 4) Каждый блок в секторе снабжен битом присутствия. Если бит присутствия равен единице, то в данный момент этот блок находится в КЭШ, если равен нулю, то блок отсутствует.
- 5) При обращении к ОП ЦП выдает физический (исполнительный) адрес.



Алгоритм определения наличия информации в КЭШ:

1. Старшие разряды исполнительного адреса сравнивают с содержимым РНПС.

В том регистре, где произошло совпадение, будет означать, что адресуемый сегмент находится в КЭШ и он расположен в секторе, соответствующему этому сегменту.

Если старшие разряды исполнительного адреса не совпали ни с одним из регистров РНПС, это означает, что адресуемый сегмент отсутствует в КЭШ.

Чтобы подкачать адресуемый сегмент в КЭШ предварительно должен быть вытеснен один из сегментов, находящихся в КЭШ.

После определения номера вытесняемого сегмента из некоторого сектора при признаковом обмене для всех блоков анализируется бит изменения. Те блоки, для которых бит изменения =1, переписываются в ОП. Далее все биты

присутствия и изменения для всех блоков сбрасываются в ноль. При сквозной записи вытеснение сегмента состоит только в обнулении битов присутствия. Подкачка адресуемого сегмента в КЭШ заключается только в занесении номера адресуемого сегмента на регистр РНПС.

2. Выделяются средние разряды. Для соответствующего блока уже в найденном секторе анализируется бит присутствия.

- Если бит присутствия = 1, то это означает, что адресуемая информация хранится в КЭШ.
- Если бит присутствия = 0, то это означает, что адресуемого блока в КЭШ нет, происходит обращение к ОП. Адресуемый блок подкачивается в КЭШ и бит присутствия для него устанавливается в единицу.

3. ЦП обращается в КЭШ по чтению или по записи информации. При обращении по чтению информация не меняется, поэтому после обращения информация в КЭШ и ОП идентична.

При записи информация в КЭШ изменяется, поэтому существуют две дисциплины работы:

1. Сквозная запись. Информация одновременно пишется в КЭШ и ОП. Вытеснение сегмента состоит только в обнулении битов присутствия

2. Признаковый обмен. Каждый блок снабжен дополнительно битом изменения. При записи информации в соответствующий блок КЭШ бит изменения устанавливается в единицу = > блок КЭШ и соответствующий блок ОП различны. Этот признак анализируется при вытеснении сегмента.

Частным случаем секторного способа является буфер прямого соответствия, когда число секторов равно единице.

10. Группо-ассоциативный способ отображения ОП на БП.

Для организации функционирования группо-ассоциативного буфера также, как и в случае секторного буфера, все поле памяти разбивается на блоки равной величины, содержащие по L байтов.

Блоки объединяются в зоны по N блоков, которые называются сегментами и секторами, соответственно, для ОП и БП. В группо-ассоциативном буфере j-й блок ($j=0..N-1$), i-го сегмента ОП ($i=0..C-1$, где C-число сегментов), может быть помещен на место j-го блока k-го сектора БП ($k=0..S-1$, где S-число секторов), однако в одном секторе буфера могут находиться блоки различных сегментов ОП. Поэтому каждому блоку придается адресная часть, в которой указывается, какому сегменту ОП принадлежит данный блок. Кроме того, при использовании признакового обмена каждый блок в БП содержит дополнительно бит изменения. Бит изменения блока равен 1, если содержимое блока изменялось, в противном случае бит изменения равен 0.

Номер сегмента, номер блока и номер байта внутри блока определяются по исполнительному адресу так же, как для секторного буфера.

Функционирование.

1. При обращении к памяти по исполнительному адресу определяется номер блока.
2. В БП просматриваются адресные части блоков с выделенным номером и анализируются номера сегментов.
 - Если номер сегмента, заданный исполнительным адресом, совпадает с номером сегмента, заданным адресной частью блока в БП, то это означает, что адресуемый блок хранится в БП. При наличии адресуемого блока в БП в зависимости от кода операции происходит чтение информации из БП в ЦП или запись информации в БП из ЦП по адресу, который определяется младшими разрядами исполнительного адреса.
 - Если адресуемый блок отсутствует в БП, необходимо вытеснить один из блоков с таким же номером, что и адресуемый блок, но принадлежащий другому сегменту. Вытесняемый блок определяется на основании принятого алгоритма вытеснения, аналогичного алгоритму вытеснения сегмента для секторного буфера.

Рассмотрим дисциплину признакового обмена:

1. Для блока, подлежащего вытеснению, проверяется бит изменения. Если бит изменения равен 1, то это означает, что содержимое блока было изменено, блок переписывается в ОП, а бит изменения сбрасывается в 0.
2. На место вытесняемого блока из ОП записывается адресуемый блок, а в адресную часть блока заносится номер адресуемого сегмента. В том случае, если для вытесняемого блока бит изменения был равен 0, то на место этого блока сразу из ОП перезаписывается адресуемый блок, а в адресную часть заносится номер адресуемого сегмента. Дальнейшая работа с буфером осуществляется по схеме, рассмотренной выше.

Так же, как и в секторном буфере, в группно-ассоциативном буфере, кроме признакового обмена, могут использоваться другие обменные дисциплины. При использовании сквозной записи данные в БП и ОП записываются параллельно. В связи с этим в группно-ассоциативном буфере не используются биты изменения. При сквозной записи в случае отсутствия адресуемого блока на место вытесняемого блока сразу же записывается адресуемый блок, а в адресную часть блока заносится номер адресуемого сегмента.

11. Основные узлы ЦП и их назначение.

Центральный процессор (ЦП) – это устройство, которое предназначается для непосредственной обработки данных и работает под управлением программ.

Основные узлы центрального процессора:

1) **Арифметико-логическое устройство (АЛУ)**. Предназначается для выполнения арифметических и логических операций над данными.

2) **Устройство управления (УУ)** – предназначается для выработки управляющих сигналов, под воздействием которых выполняются машинные команды.

3) **Регистры общего назначения (РОНы)**. РОНы - это программно-адресуемые регистры, предназначаются для хранения операндов, результатов, а также для хранения индекса, базы, используются при вычислении адреса.

4) **Управляющие регистры.**

Счетчик команд (СК) – хранит адрес следующей выполняемой команды.

Регистр команд (РК) – хранит текущую выполняемую команду.

5) **Вспомогательные блоки** – к ним относится блок прерывания, блок связи центрального процессора (ЦП) и оперативной памяти (ОП), блок контроля и диагностики и т.д.

12. Команды ЭВМ. Кодирование команд. Адресность команд.

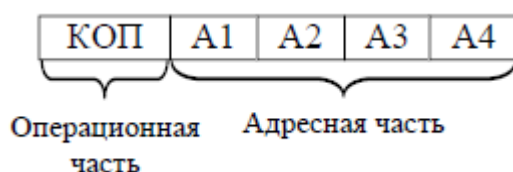
Машинная команда – это двоичный код, который включает в себя **операционную часть** и **адресную часть**. В адресной части содержится информация об адресах операндов и результатов. В операционной кодируется код операции

Процесс выполнения команды состоит из выполнения **микрокоманд**, каждая микрокоманда включает в себя одну или несколько **микроопераций**. Каждой микрооперации соответствует определенный **управляющий сигнал**.

Различаются следующие команды:

- 4-х адресные,
- 3-х адресные,
- 2-х адресные,
- одноадресные,
- безадресные.

4-х адресные команды



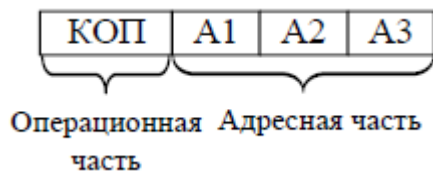
Поле A1 – информация об адресе первого операнда.

Поле A2 – информация об адресе второго операнда.

Поле A3 – информация об адресе результата.

Поле A4 – информация об адресе следующей выполняемой команды.

3-х адресные команды



В 3-х адресной команде отсутствует A4. При использовании 3-х адресной команды добавляется дополнительное устройство – счетчик команд и дополнительно появляются команды перехода.

Адрес следующей команды хранится на счетчике команд.

2-х адресные команды

Поле A3 – отсутствует. **Результат записывается на место 1-го или 2-го операнда.**

При этом операнд затирается и если его необходимо использовать в дальнейшем, он предварительно должен быть сохранен.

Одноадресные команды

Есть только поле A1. В структуру ЦП вводится дополнительный **регистр аккумулятора (РА)**, на который предварительно дополнительной командой засылается **второй операнд**. **Результат записывается на место либо первого операнда, либо в аккумулятор.**

Безадресные команды

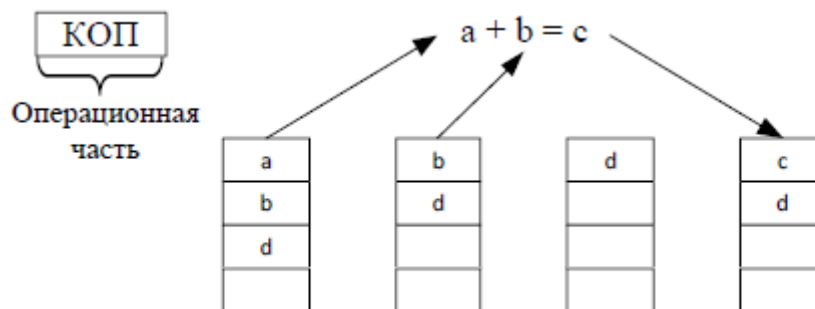


Рис. 18. Структура безадресной команды

Имеется только поле кода операции. Стек используется для хранения операндов и записи результата.

Этапы выполнения команды:

1. Считали команду из памяти, перенесли ее в процессор, переместили на регистр команд
2. Дешифрация кода операции
3. Берем адрес, подаем его в память, подаем сигнал чтения, подаем 1-ый операнд на регистр АЛУ. Остальные операнды так же. Потом в сумматор
4. Из центрального процессора сохраняем результат в память. Для этого подаем на РАП адрес, а в РЧП результат и сигнал записи.

или:

1 этап: выбор команды из памяти {чтение адреса следующей команды и передача этой команды на регистр команд}

2 этап: дешифрация кода операции

3 этап: формирование исполнительного адреса и выбор операндов {чтение адресов операндов из памяти и передача операндов на входные регистры алу}

4 этап: выполнение операции в АЛУ

5 этап: запись результата

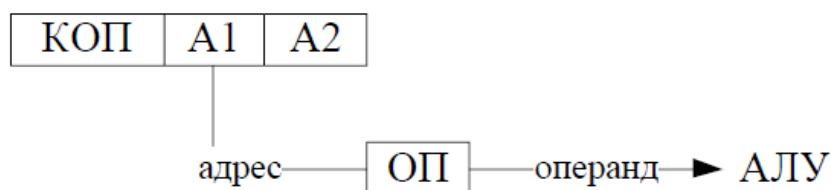
13. Способы адресации.

Адресный код в команде – это информация об адресе.

Исполнительный адрес операнда – это физический адрес ячейки памяти, в которой хранится операнд или в которую необходимо записать результат.

Рассмотрим варианты адресаций на примере 2-ух адресной команды.

Прямая адресация

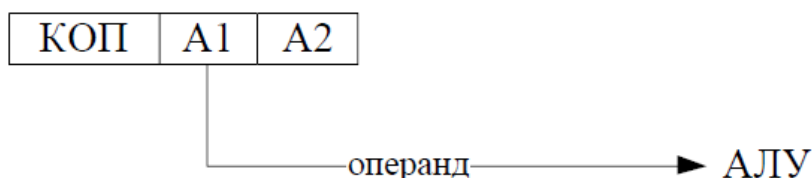


В адресном поле команды A1 задаётся исполнительный адрес операнда.

Выбор операнда $\begin{cases} \text{РАП:}=\text{РК}(A1) \\ \text{РЧП:}=\text{ЧТ}(\text{РАП}) \\ \text{РА:}=\text{РЧП} \end{cases}$

РА – входной регистр АЛУ.

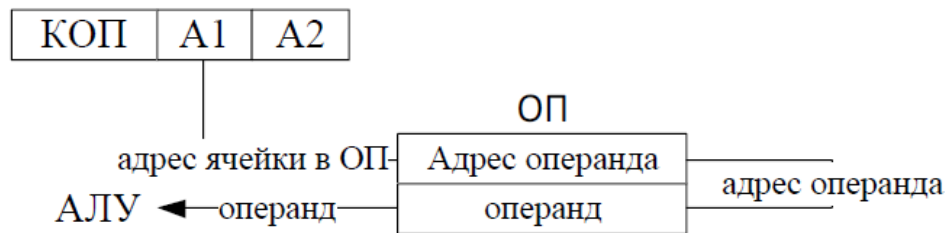
Непосредственная адресация



В адресном поле команды задается непосредственно сам операнд.

$\text{РА:}=\text{РК}(A1)$

Косвенная адресация



В адресном поле команды задается адрес ячейки оперативной памяти (ОП), в которой хранится адрес операнда. Иначе эта адресация называется адрес адреса.

РАП:=РК(A1)

РЧП:=ЧТ(РАП) – выбор адреса операнда из ЗМ

РАП:=РЧП

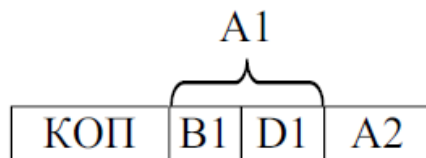
РЧП:=ЧТ(РАП) – выбор операнда из ЗМ

РА:=РЧП

Регистровая адресация

[<См. вопрос 15>](#)

Базовая адресация

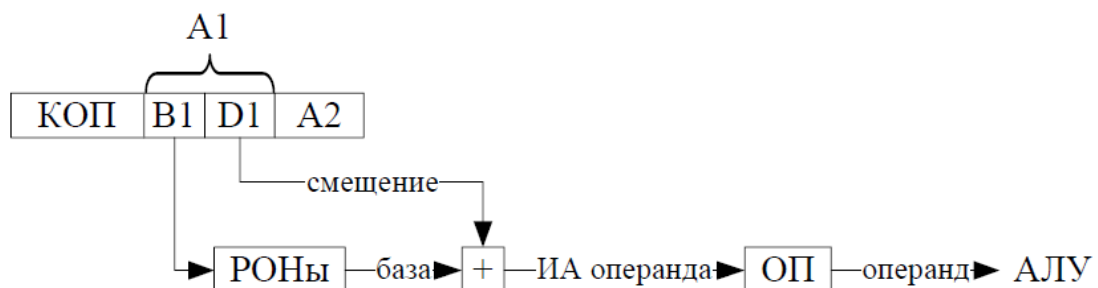


B1 – номер базового регистра, (B1) – база.

D1 – смещение относительно базы.

Исполнительный адрес не задан явно в адресном поле.

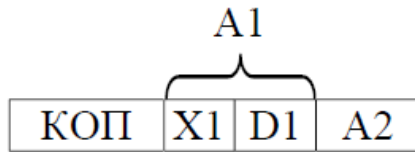
Исполнительный адрес $A1 = (B1) + D1$.



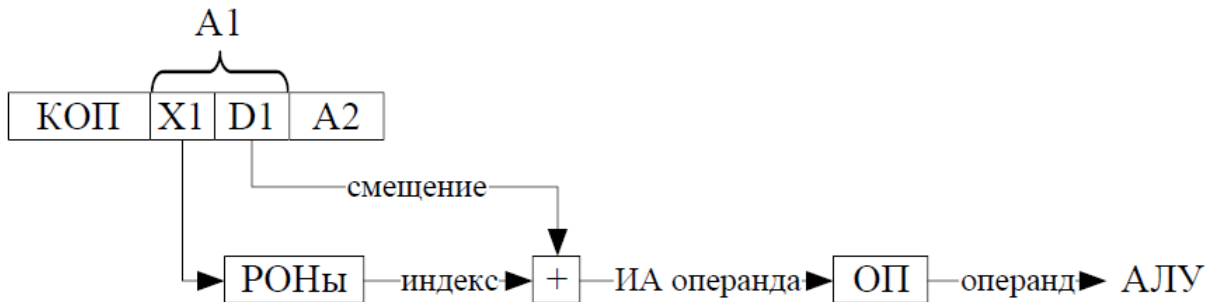
Базовая адресация обеспечивает перемещаемость программ в памяти.

При базовой адресации операнды хранятся в ОП. Для вычисления исполнительного адреса операнда необходимо к содержимому базового регистра прибавить смещение, которое выбирается из адресного поля команды.

Индексная адресация



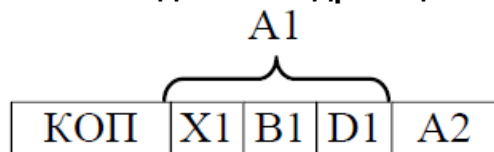
X1 – номер индексного регистра, (X1) – индекс.
Исполнительный адрес $A1 = (X1) + D1$.



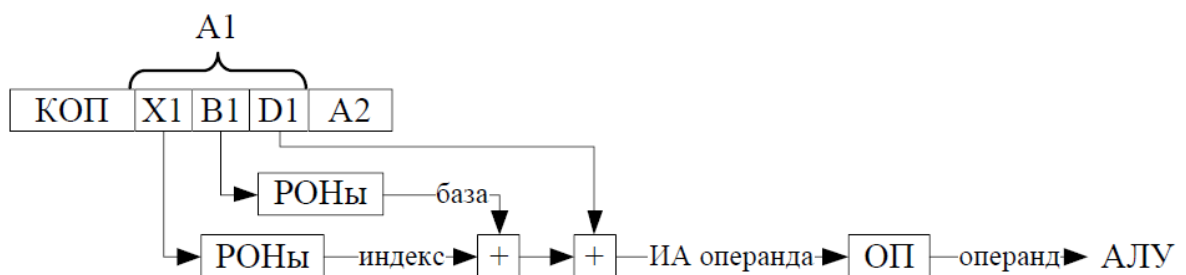
Индексная адресация используется при работе с массивами, таблицами.

Исполнительный адрес операнда вычисляется аналогично базовой адресации.

Базово-индексная адресация

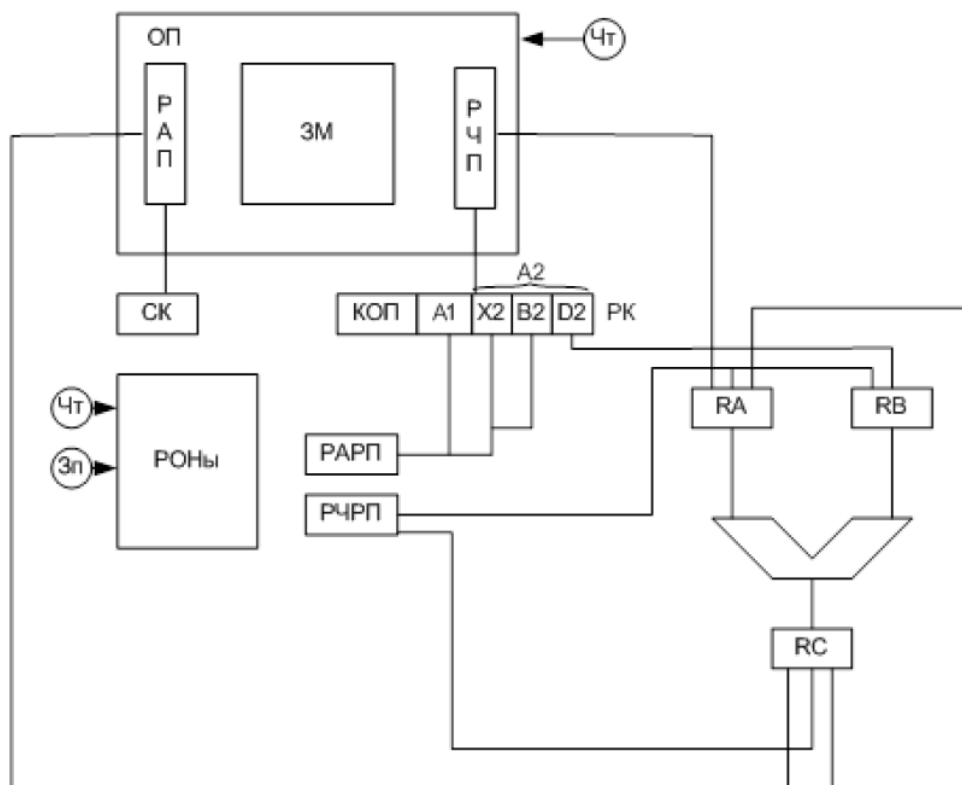


(X1) – индекс (содержимое индексного регистра).
(B1) – база (содержимое базового регистра).
D1 – смещение (выбирается из команды и подается на АЛУ).
Исполнительный адрес $A1 = (X1) + (B1) + D1$.



1-ый операнд: регистровая адресация.

2-ой операнд: базово-индексная адресация.



1 этап. Выбор команды из памяти:

$$РАП:=СК$$
$$P_{\text{ЧП}} := \chi_T(P_{\text{АП}})$$
$$PK := P4 \Pi$$
$$CK := CK + n$$

2 этап. Дешифрация кода операции.

3 этап. Формирование ИА и выбор операндов:

Выбор индекса X_2 { $РАРП:=PK(X_2)$ – номер индексного регистра подается в РОНЫ
 $РЧРП:=ЧТ(РАРП)$ – выбор индекса

Выбор
индекса y_2 } $PЧРП := Ч_T(PАРП)$ – выбор индекса

индекса X2 RA:=PЧРП - пересылка индекса на входной регистр АЛУ

Выбор { РАРП:=РК(В2) – номер базового регистра подается в РОНЫ

Выбор
индекса В? } РЧРП:=ЧТ(РАРП) – выбор базы

индекса ВЗ [RB:=РЧРП - пересылка базы на входной регистр АЛУ

RC:=RA + RB – вычисление X2 + B2

RA:=RC – на RA пересылается часть ИА

RB:=PK(D2) – выбор смещения

RC:= RA + RB – вычисление ИА

РАП:=RC – ИА подается на РАП

РЧП:=ЧТ(РАП) – выбор 2-го операнда

RA:=PCП – пересылка второго операнда на входной регистр АЛУ

Выбор 1-го [РАПП:=РК(А1) - на РАПП подается номер регистра, где хранится 1-ый операнд

РЧРП:=ЧТ(РАРП) - выбор первого операнда из РОНов

операнда	RV:=PЧРП - пересылка первого операнда на входной регистр АЛУ
----------	--

4 этап. Выполнение операции в АЛУ:

$RC := RA$ – операция RB

5 этап. Запись результатов:

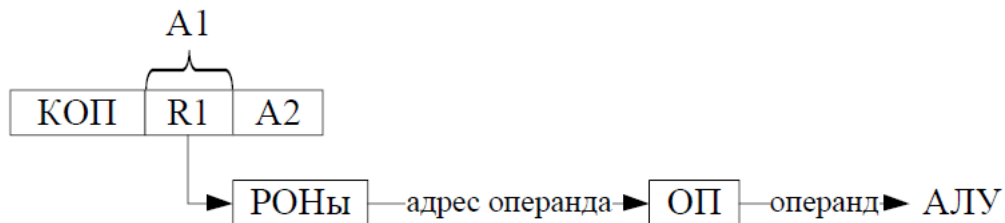
$РЧРП := RC$

$Зп(РАРП) := РЧРП$

Косвенно-регистрая адресация



В адресном поле команды указывается номер РОНа, который содержит адрес операнда, хранящийся в ОП (оперативной памяти).



$РАРП := РК(R1)$

$РЧРП := Чт(РАРП)$ - выбор адреса операнда из РОНов

$РАП := РЧРП$

$РЧП := Чт(РАП)$ - выбор операнда из ОП

$RA := РЧП$

Если нужно записать результат на место операнда с косвенно-регистрая адресацией, то результат записывается в ОП, а не в РОНЫ.

14. Влияние на работу ЦП адресности команд и способов адресации. Микропрограмма выполнения команд регистр-регистр.

<Влияние на работу ЦП адресности команд и способов адресации: см. вопросы 12-13>

<Микропрограмма выполнения команд регистр-регистр: см. вопрос 15>

15. Организация работы ЦП и ОП. Микропрограмма выполнения команд формата регистр-регистр.

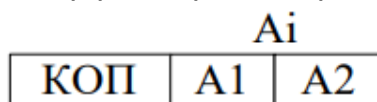
<Организация работы ЦП и ОП: см. вопросы 11-12>

Регистровая адресация

В структуру ЦП дополнительно вводятся *регистры общего назначения (РОНы)*. Они вводятся для того, чтобы:

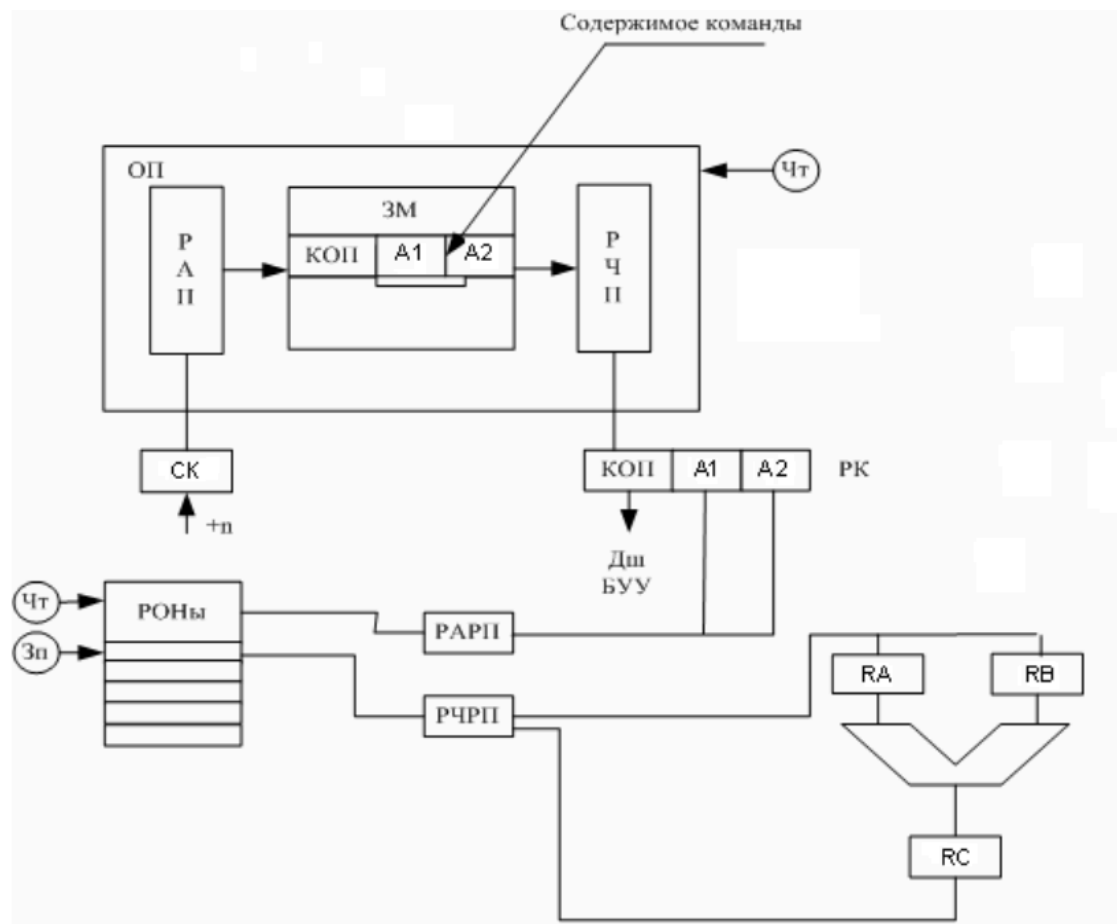
1. Сократить время выбора операндов по сравнению с ОП.
2. Сократить размер поля A_i для кодирования номера РОНа.

При использовании регистровой адресации в адресном поле команд A_i задается номер регистра, где хранится операнд:



При написании микропрограммы принимается, что запись результата производится на место первого операнда.

Структурная схема операционной части ЦП при выполнении 2-х адресной команды формата регистр-регистр (R-R) (где РК – *регистр команд* – предназначается для хранения текущей выполняемой задачи):



Микропрограмма выполнения 2-х адресной команды согласно структурной схеме:

1 этап. Выбор команды из памяти

$РАП := СчК$ - адрес следующей команды подается на РАП

$РЧП := Чт(РАП)$ - выбор команды из ЗМ

$ПК := РЧП$ - машинная команда из памяти передается на ПК в ЦП

$СчК := СчК + n$ - n-длина выполняемой машинной команды в байтах

2 этап. Дешифрация кода операции

3 этап. Формирование исполнительного адреса и выбор операндов.

Операнды находятся в РОНах

Выбор 1-го операнда	{	$РАП := ПК(R1)$ - на РАП из ПК подается номер регистра, где хранится 1-ый операнд
		$РЧРП := Чт(РАП)$ - выбор первого операнда из РОНов
		$R1 := РЧРП$ - пересылка первого операнда на входной регистр АЛУ
Выбор 2-го операнда	{	$РАП := ПК(R2)$ - на РАП из ПК подается номер регистра, где хранится 2-ой операнд
		$РЧРП := Чт(РАП)$ - выбор второго операнда из РОНов
		$R2 := РЧРП$ - пересылка второго операнда на входной регистр АЛУ

4 этап. Выполнение операции в АЛУ

PC:=P1 операция P2
5 этап. Запись результата
PAP:=PK(R1)
PЧРП:=PC
Зп(PAP):=PЧРП

16. Принцип работы блока УУ.

Устройство управления центрального процессора вырабатывает последовательность функциональных сигналов, необходимую для выполнения команды. Команда в процессоре реализуется, обычно, за несколько тактов, в каждом из которых выполняется одна или несколько микроопераций.

Операционная часть выполняет заданную **микропрограмму**, состоящую из микрокоманд.

Микрокоманда включает в себя одну или несколько микроопераций.

Микрооперация – это элементарная функциональная операция, выполняемая под воздействием одного управляющего сигнала в течение одного такта.

Устройство управления (УУ) служит для выработки последовательности управляющих сигналов, под воздействием которых выполняются микрооперации.

Классификация микропрограммных устройств управления

1. Горизонтальные, вертикальные, смешанные.

В смешанных все микрооперации разбиваются на группы. В одной микропрограмме можно кодировать параллельно микрооперации из разных групп. Внутри каждой группы кодируется номер управляющего сигнала

2. Однофазные и многофазные

В однофазных зависимые микрооперации должны быть разнесены по разным тактам

В многофазных такт делится на подтакты (фазы). Зависимые микрооперации могут выполняться в одном такте, если они разнесены по разным фазам

3. Статические и динамические

В статических нельзя менять код микропрограммы, а в динамических можно

17. Микропрограммная реализация УУ. Классификация микропрограммных УУ.

Микропрограммный принцип организации управления заключается в том, что каждой микрооперации ставится в соответствие слово (или часть слова), называемая микрокомандой и хранимое в памяти подобно тому, как хранятся в памяти команды обычной пользовательской программы.

При таком подходе команде соответствует микропрограмма, то есть набор микрокоманд, указывающих, какие функциональные импульсы и в какой последовательности необходимо возбуждать для выполнения данной операции.

Этот подход получил название **микропрограммный** или "хранимой логики". Микропрограммное устройство управления может быть реализовано следующими способами:

- горизонтальный способ;
- вертикальный способ;
- горизонтально - вертикальный;
- вертикально - горизонтальный.

Горизонтальное микропрограммирование

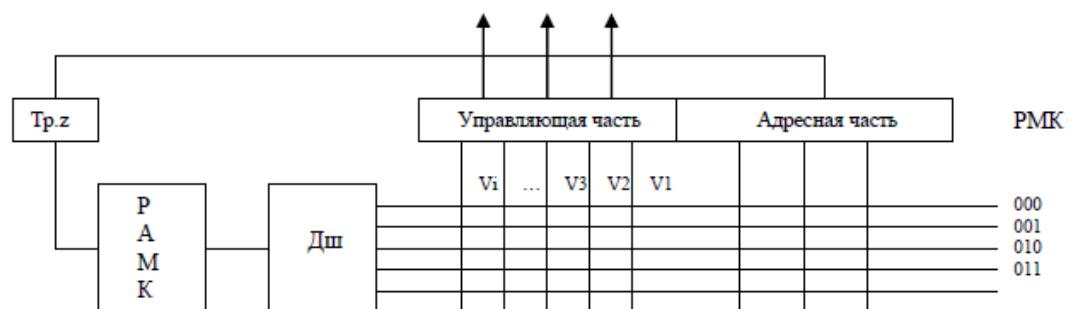


Рис. 24. Схема Уилкса

Каждая микрокоманда состоит из **двух частей**: **управляющей** части, где кодируются управляющие сигналы V_i , и **адресной** части.

В адресной части микрокоманды кодируется адрес ячейки памяти, где хранится следующая выполняемая микрокоманда.

В начальный момент времени на РАМК подается адрес ячейки памяти, где хранится первая микрокоманда. По этому адресу из памяти микрокоманда считывается и подается на регистр микрокоманд.

Из управляющей части микрокоманды управляющие сигналы подаются на вентили в операционную часть ЦП, а из адресной части на регистр адреса микрокоманд в следующем такте заводится адрес следующей выполняемой микрокоманды.

При горизонтальном микропрограммировании для каждого управляющего сигнала в управляющей части выделяется **отдельный разряд**. При такой организации можно совмещать микрооперации во времени.

Достоинство: **высокое быстродействие** за счёт совмещения микроопераций во времени.

Недостатки: **требуется большая ёмкость памяти** для хранения микрокоманды, так как реально совместить большое количество микроопераций в одной микрокоманде не удаётся.

Вертикальное микропрограммирование

При вертикальном микропрограммировании в операционной части МК кодируется номер управляющего сигнала, поэтому на выходе регистра микрокоманд в управляющей части ставится дешифратор.

Достоинство: значительно сокращается емкость памяти микрокоманд.

Недостатки: увеличенное время выполнения микрокоманд из-за невозможности совмещения микроопераций в микрокоманде и появления дешифратора на выходе. Одной микрокоманде соответствует всего один функциональный сигнал поэтому число микрокоманд в пакете должно соответствовать числу функциональных сигналов.

18. Схемная реализация УУ.

При горизонтальном микропрограммировании для каждого управляющего сигнала в управляющей части выделяется отдельный разряд. Если в некотором такте управляющий сигнал должен быть равен 1, то в соответствующем разряде записывается 1. Таким образом, количество разрядов в управляющей части соответствует числу управляющих сигналов. При такой организации можно совмещать микрооперации во времени.

Достоинство: высокое быстродействие за счёт совмещения микроопераций во времени.

Недостатки: требуется большая ёмкость памяти для хранения микрокоманды, так как реально совместить большое количество микроопераций в одной микрокоманде не удаётся.

Схема Вилкса

Основой является память микрокоманд. Горизонтальные линии – ячейки памяти, где хранится одна микрокоманда (выполняется за 1 такт). По вертикали – значения вентилей.

Номера управляющих сигналов, которые подаются в операционную часть ЦП, кодируются в управляющей части. В адресной части кодируется адрес ячейки памяти, где содержится следующая выполняемая микрокоманда.

На регистр адреса МК подается адрес, в соответствии с которым выбирается МК и подается на регистр МК. Из адресной части МК адрес следующей выполняемой МК будет подан на ПАМК через линию задержки. Из управляющей части управляющие сигналы подаются на вентили и определяется порядок выполнения МК.

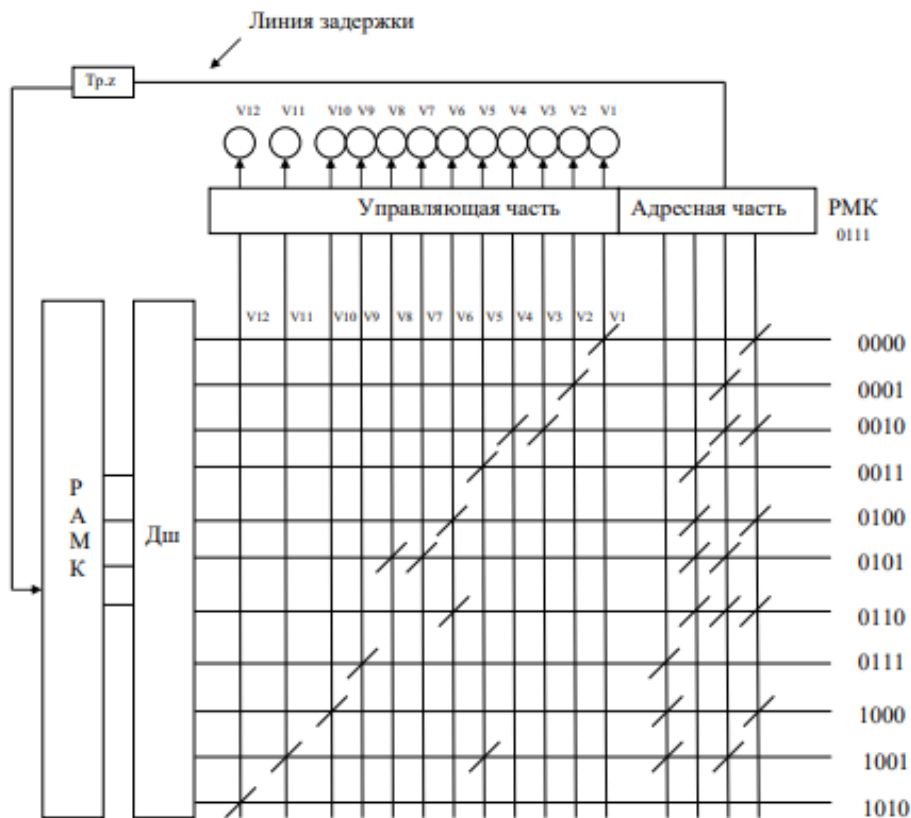


Рис. 26. Пример реализации горизонтального микропрограммного устройства управления

При **вертикальном микропрограммировании** в операционной части МК кодируется номер управляющего сигнала, поэтому на выходе регистра микрокоманд в управляющей части ставится дешифратор.

Достоинство: значительно **сокращается емкость памяти** микрокоманд.

Недостатки: **увеличенное время выполнения** микрокоманд из-за невозможности совмещения микроопераций в микрокоманде и появления дешифратора на выходе.

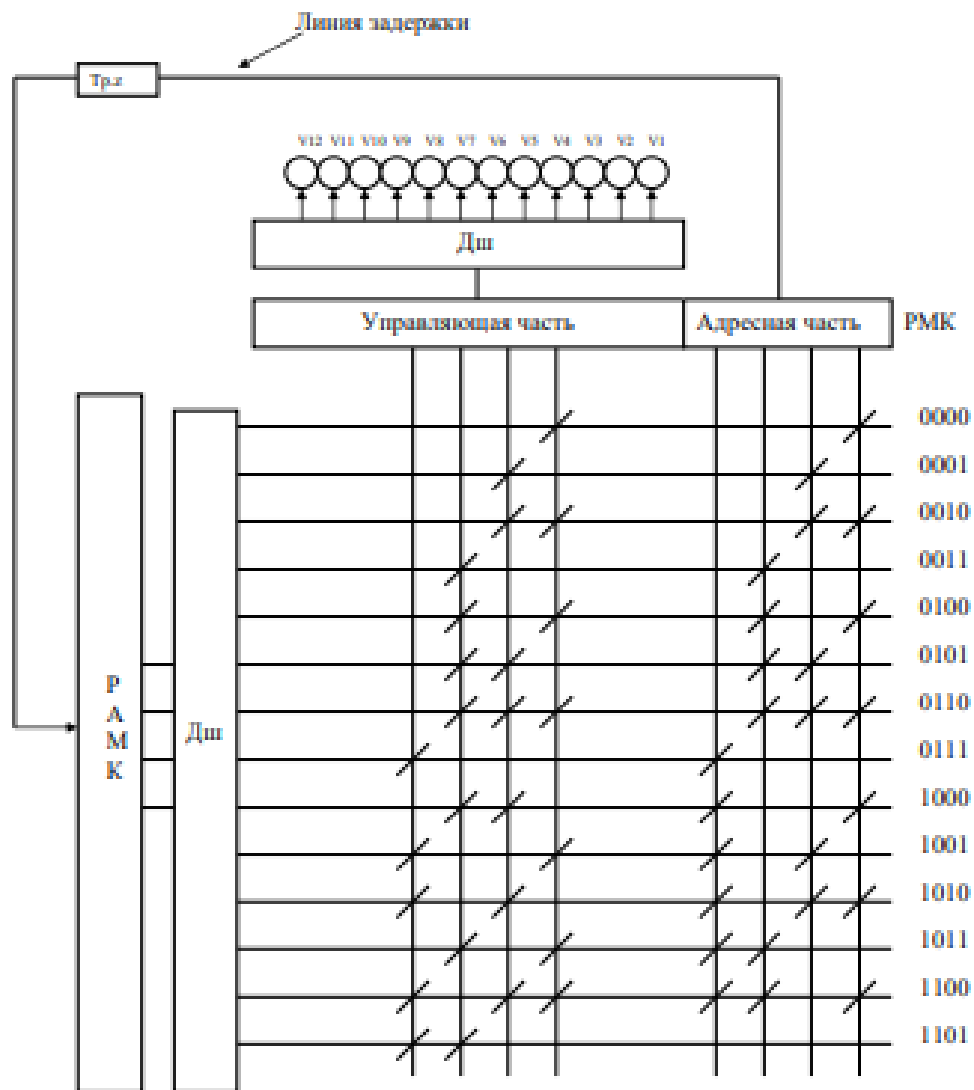
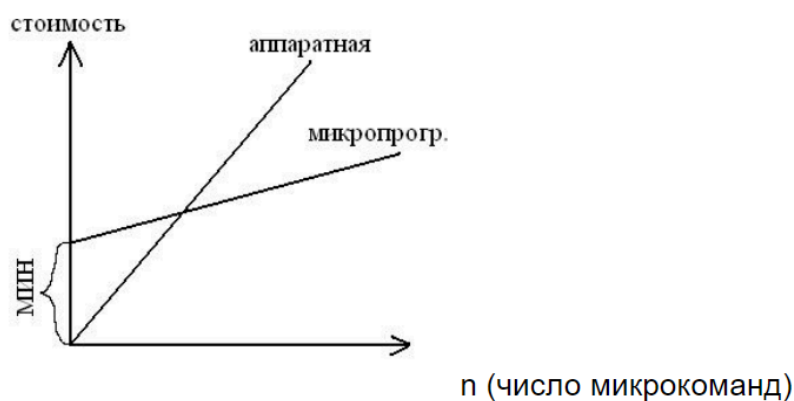


Рис. 27. Пример реализации вертикального микропрограммного устройства управления

19. Сравнение микропрограммной и схемной реализации УУ.



1. **Стоимость:** Чем **больше** микрокоманд, тем **выгоднее** использовать микропрограмму УУ. Микропрограммная реализация всегда дешевле.

2. **Быстродействие:** Аппаратная реализация **более быстрая**.
3. **Надёжность:** У микропрограммной реализации **надёжность выше**, чем у аппаратной.
4. **Сроки проектирования:** У микропрограммной реализации сроки проектирования **сокращаются**.

20. АЛУ – назначение, классификация. Выполнение операций сложения, вычитания над числами с фиксированной точкой.

Арифметико-логическое устройство (АЛУ) – предназначается для выполнения арифметических и логических операций над данными.

В АЛУ выполняются операции с фиксированной точкой; с плавающей точкой; операции двоично – десятичной арифметики, логические операции, обработка алфавитно-цифровой информации.

АЛУ бывают:

- Одноразрядными и многоразрядными
- Однофункциональными и многофункциональными
- Синхронными и асинхронными
- Последовательными, параллельными и последовательно-параллельными

Моделирование функционирования АЛУ при выполнении операции сложения с использованием обратного кода

Чтобы получить обратный код отрицательного двоичного числа, необходимо значащие разряды проинвертировать, а в знаковый разряд поставить 1.

Положительные числа – в прямом коде, отрицательные числа – в обратном (производится суммирование кодов чисел, включая знаковые разряды).

При возникновении 1 переноса из знакового разряда она подсуммируется к младшему разряду результата.

Примеры:

Зн.	Инф.
	разр.
0	0101
1	1011
1 0	0000
	1
0	0001

единица переноса прибавляется к младшему разряду

В знаковом разряде результата 0, следовательно, результат положительный и представлен в прямом коде.

Зн.	Инф.
	разр.
1	1010
0	0100
1	1110

В знаковом разряде результата 1, следовательно, результат отрицательный и представлен в обратном коде. Прямой код результата $C_{пр}=10001_{(2)}=-1_{(10)}$.

Моделирование функционирования АЛУ при выполнении операции сложения с использованием дополнительного кода

Чтобы получить дополнительный код отрицательного двоичного числа, необходимо прибавить 1 к обратному коду.

Положительные числа – в прямом коде, отрицательные числа – в дополнительном (производится суммирование кодов чисел, включая знаковые разряды).

При возникновении 1 переноса из знакового разряда она отбрасывается (в этом и состоит преимущество дополнительного кода над обратным – не нужно учитывать единицу переноса из знакового разряда в процессе сложения).

Примеры:

Зн.	Инф.
	разр.
0	0101
1	1100
0	0001

единица переноса отбрасывается

В знаковом разряде результата 0, следовательно, результат положительный и представлен в прямом коде.

Зн.	Инф. разр.
1	1011
0	0100
1	1111

В знаковом разряде результата 1, следовательно, результат отрицательный и представлен в дополнительном коде. Прямой код результата $C_{пр} = 10001_{(2)} = -1_{(10)}$.

Алгоритм алгебраического сложения/вычитания двоичных чисел с фиксированной запятой при использовании дополнительного кода для представления слагаемых.

При выполнении операции сложения положительные слагаемые представляются в прямом коде, а отрицательные – в дополнительном. Производится сложение двоичных кодов, включая разряды знаков. Для отслеживания переполнения существует 2 основных метода:

- по наличию и отсутствию переноса;
- использование модифицированного кодирования.

В первом случае, если возникает перенос из знакового разряда суммы при отсутствии переноса в этот разряд или перенос в знаковый разряд при отсутствии переноса из разряда знака, то имеется переполнение разрядной сетки. Если нет переносов из знакового разряда и в знаковый разряд суммы или есть оба переноса, то переполнения нет. При этом, если знаковый разряд суммы равен 0, то сумма положительна, если 1, то сумма отрицательна и представлена в дополнительном коде.

Во втором случае для представления знака отводится два или более разрядов. В случае использования двух знаковых разрядов комбинация 00 позволяет сделать вывод о том, что результат получился положительным, 11 – отрицательным, 10 или 01 – переполнение. Модифицированный код позволяет ЭВМ в автоматическом режиме (а не человеку вручную) отслеживать переполнение.

Пример без переполнения:

Рассмотрим пример операции сложения: $C=A+B$. В качестве слагаемых возьмем 5-разрядные числа (4 информационных разряда и 1 разряд – знаковый): $A=11$, $B=-4$. В двоичной системе счисления положительные числа представляются в прямом коде, а отрицательные – в дополнительном. Следовательно, $A=01011$ и $B=11100$. Производим операцию сложения:

Зн.	Инф.
	разр.
0	1011
1	1100
0	0111
	

Переполнения разрядной сетки не было, так как был перенос и в знаковый разряд, и из знакового разряда.

Так как знак результата равен 0, то получаем положительный результат в прямом коде $C=00111_{(2)}=7_{(10)}$.

Пример с переполнением:

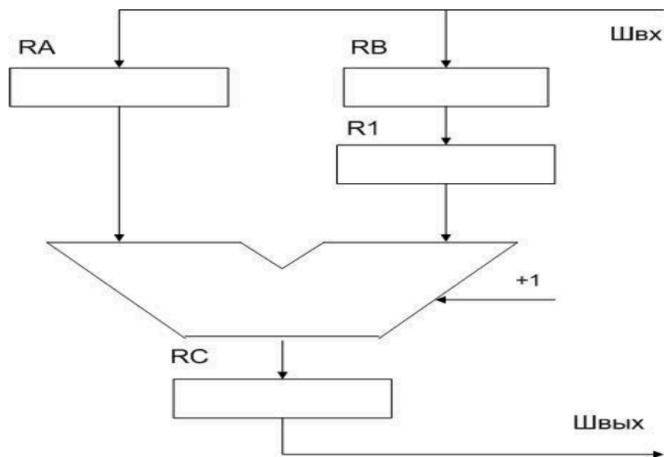
Рассмотрим пример операции вычитания: $C=A-B$. В качестве уменьшаемого и вычитаемого возьмем 5-разрядные числа (4 информационных разряда и 1 разряд – знаковый): $A=11_{(10)}=01011_{(2)}$, $B=-6_{(10)}=11010_{(2)}$.

Для выполнения операции вычитания берем дополнительный код вычитаемого. Получаем, что $B=00110$. Производим операцию сложения:

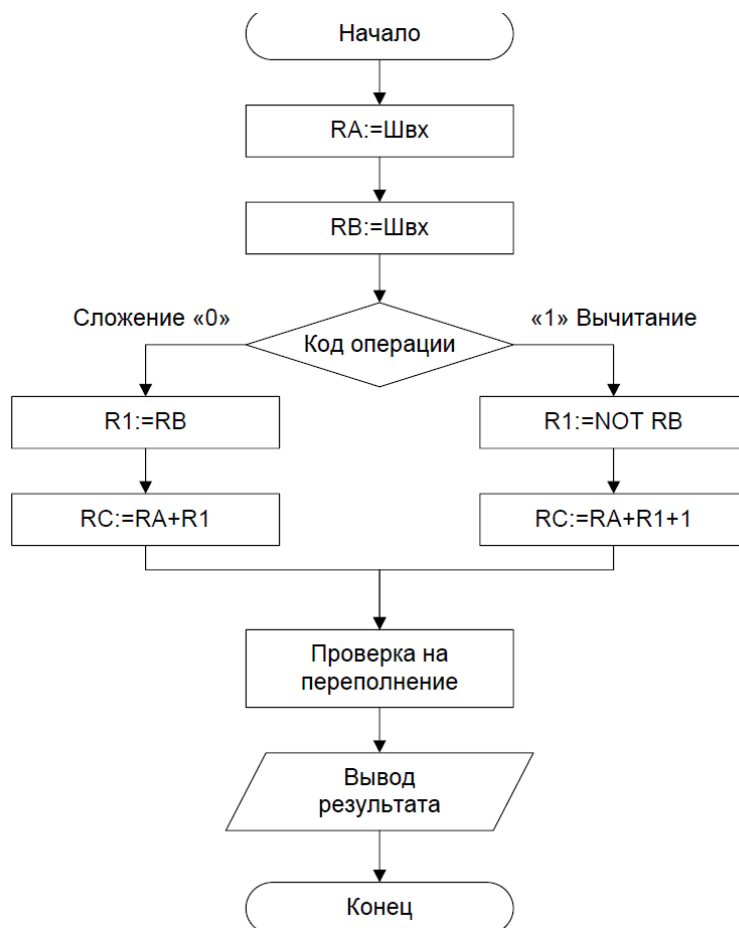
Зн.	Инф.
	разр.
0	1011
0	0110
1	0001
	

Так как был перенос в знаковый разряд, и не было переноса из знакового разряда, то произошло переполнение разрядной сетки.

Структурная схема АЛУ для операций сложения/вычитания:



Блок-схема выполнения операций сложения/вычитания:



I – код операции: 0 – сложение, 1 – вычитание

INPA, INPB – регистры для ввода операндов;

RA – первый операнд АЛУ;

RB – второй операнд АЛУ;

R1 – второй операнд в обратном коде;

RC – результат операции в АЛУ.

21. Алгоритмы выполнения операций умножения. АЛУ для выполнения операций умножения над числами с фиксированной точкой.

Особенности выполнения операции умножения чисел с фиксированной запятой, представленных в прямом и дополнительном коде

В каждом цикле выполнения операции умножения анализируется очередная цифра множителя.

Если очередная цифра множителя равна 1, то к сумме частичных произведений прибавляется множимое, в противном случае прибавляется нуль.

Цикл завершается сдвигом множимого относительно суммы частичных произведений либо сдвигом суммы частичных произведений относительно неподвижного множимого.

Таким образом, выполнение операции умножения в АЛУ сводится к последовательности операций сложения и сдвига.

В случае отрицательного операнда при умножении чисел, представленных в прямом коде, операция умножения сводится к выполнению следующих этапов:

- 1) определение знака произведения путем сложения по модулю 2 знаковых разрядов множимого и множителя;
- 2) обнуление знаковых разрядов отрицательных операндов;
- 3) выполнение операции умножения чисел, представленных в прямом коде.

Алгоритм умножения двоичных чисел с фиксированной запятой, представленных в дополнительном коде

1. Сомножители представляются в дополнительном коде. Так как дополнительный код положительного числа совпадает с прямым кодом, то положительные числа будут представлены в прямом коде.
2. Исходное значение суммы частичных произведений принимается равным 0.
3. Если анализируемая цифра множителя равна 1, то к сумме частичных произведений прибавляется множимое в том коде, в котором оно представлено.

Прибавление производится с выравниванием складываемых чисел по старшим разрядам.

Если анализируемая цифра множителя равна 0, прибавление не производится.

4. Сумма частичных произведений сдвигается на один разряд вправо.

При этом, если сумма отрицательна, осуществляется модифицированный сдвиг.

5. Пункты 3 и 4 последовательно выполняются для всех цифровых разрядов множителя, начиная с младшего.

6. Если множитель – **положительное число**, полученный результат представляет собой **произведение**.

Если множитель **отрицателен**, то для получения произведения к результату **прибавляется множимое с обратным знаком**. При этом прибавление производится с выравниванием складываемых чисел по старшим разрядам.

Если результат размещается в **двойном слове**, то он предварительно **сдвигается на один разряд вправо**.

7. Произведение получается в **прямом** коде, если сомножители имеют **одинаковые** знаки.

И произведение получается в **дополнительном**, если сомножители имеют **разные** знаки.

Способы умножения чисел с фиксированной запятой

1. Умножение чисел, представленных в прямом коде, начиная с младших разрядов множителя, со сдвигом суммы частичных произведений вправо и при неподвижном множимом.

$$C=A*B, \text{ где } A=14_{(10)}=01110_{(2)}, B=4_{(10)}=00100_{(2)}$$

Множитель				
Зн.	Информ. разряды			
4-й	3-й	2-й	1-й	0-й
0	0	1	0	0

Зн.	Информ. разряды	Описание действий
0	1110	Множимое (мн-ое)
0	0100	Множитель (мн-ль)
0	0000	Сумма частичных произведений (СЧП)
0	0000	Сдвиг СЧП на 1 разряд вправо (0-й разряд множителя =0)
+0	0000	Сдвиг СЧП на 1 разряд вправо (1-й разряд множителя =0)
0	1110	Множимое
0	1110	СЧП (2-й разряд множителя =1)
0	0111	Сдвиг СЧП на 1 разряд вправо
0	0011	Сдвиг СЧП на 1 разряд вправо (3-й разряд множителя =0)
0	0001	Сдвиг СЧП на 1 разряд вправо (4-й разряд множителя =0)

2. Умножение чисел, представленных в прямом коде, начиная с младших разрядов множителя, со сдвигом множимого влево и при неподвижной сумме частичных произведений.

$$C=A*B, \text{ где } A=11_{(10)}=01011_{(2)}, B=-5_{(10)}=10101_{(2)}$$

Множитель				
Зн.	Информац. разряды			
4-й	3-й	2-й	1-й	0-й
0	0	1	0	1

Информ. разряды	Описание действий	
	01011	Множимое
	00101	Множитель
+	00000	Сумма частичных произведений, 0-ой разряд множителя =1
	01011	Множимое
+	01011	СЧП, Сдвиг мн-ого на 1 разряд влево, 1-й разряд множителя =0
0	0000	
+0	01011	СЧП, Сдвиг мн-ого на 1 разряд влево, 2-й разряд множителя =1
01	011	Сдвинутое множимое
+01	10111	СЧП, Сдвиг мн-ого на 1 разряд влево, 3-й разряд множителя =0
000	00	
+001	10111	СЧП, Сдвиг мн-ого на 1 разряд влево, 4-й разряд множителя =0
0000	0	
0001	10111	Результат операции умножения

3. Умножение чисел, представленных в прямом коде, начиная со старших разрядов множителя, со сдвигом суммы частичных произведений влево и при неподвижном множимом.

$C=A*B$, где $A=13_{(10)}=01101_{(2)}$, $B=9_{(10)}=01001_{(2)}$

Множитель				
Зн.	Информац. разряды			
4-й	3-й	2-й	1-й	0-й
0	1	1	0	1

Информ. разряды	Описание действий	
	01101	Множимое
	01001	Множитель
<	00000	СЧП, 4-ый разряд множителя =0
+0	00000	Сдвиг СЧП на 1 разряд влево, 3-й разряд множителя =1
	01101	Множимое
<0	01101	Сумма частичных произведений
+00	11010	Сдвиг СЧП на 1 разряд влево, 2-й разряд множителя =0
	00000	
<00	11010	Сумма частичных произведений
+001	10100	Сдвиг СЧП на 1 разряд влево, 1-й разряд множителя =0
	00000	
<001	10100	Сумма частичных произведений
+0011	01000	Сдвиг СЧП на 1 разряд влево, 0-й разряд множителя =1
	01101	Множимое
0011	10101	Результат операции умножения

4. Умножение чисел, представленных в прямом коде, начиная со старших разрядов множителя, со сдвигом множимого вправо и при неподвижной сумме частичных произведений.

$C=A*B$, где $A=11_{(10)}=01011_{(2)}$, $B=15_{(10)}=01111_{(2)}$

Множитель				
Зн.	Информац. разряды			
4-й	3-й	2-й	1-й	0-й
0	1	1	1	1

Зн.	Информ. разряды	Описание действий
0	1011	Множимое
0	1111	Множитель
0	0000	СЧП, 4-й разряд мн-ля =0, Сдвиг мн-го на 1р. вправо
+0	0000	СЧП, 3-й разряд мн-ля =1
0	0101 1	Мн-мое, сдвинутое на 1 разряд вправо
+0	0101 1	СЧП, 2-й разряд мн-ля =1
0	0010 11	Мн-мое, сдвинутое на 1 разряд вправо
+0	1000 01	СЧП, 1-й разряд мн-ля =1
0	0001 011	Мн-мое, сдвинутое на 1 разряд вправо
+0	1000 01	СЧП, 0-й разряд мн-ля =1
0	0000 1011	Мн-мое, сдвинутое на 1 разряд вправо
0	1010 0101	Результат операции умножения

Сравнение структуры АЛУ для каждого метода умножения

Элементы структуры АЛУ	Методы умножения			
	1	2	3	4
Регистр множителя	Регистр со сдвигом вправо, n-разрядный, после выполнения операции умножения содержит мл. разряды произведения	Регистр со сдвигом вправо, n-разрядный	Регистр со сдвигом влево, n-разрядный	Регистр со сдвигом влево, n-разрядный
Регистр множимого	n-разрядный регистр	Регистр со сдвигом влево, 2n-разрядный	n-разрядный регистр	Регистр со сдвигом вправо, 2n-разрядный
Сумматор частичных произведений	Регистр со сдвигом вправо, n-разрядный, после выполнения операции умножения содержит ст. разряды произведения	2n-разрядный регистр, после выполнения операции умножения содержит произведение	Регистр со сдвигом влево, 2n-разрядный, после выполнения операции умножения содержит произведение	2n-разрядный регистр, после выполнения операции умножения содержит произведение

Состав и назначение основных регистров АЛУ для выполнения операции умножения чисел с фиксированной запятой, представленных в прямом коде.

R1 – регистр для хранения множимого;

R2 – регистр для хранения множителя и формирования младшей части произведения;

RA – регистр для хранения первого операнда АЛУ (множимое, либо 0);

RB – регистр для хранения второго операнда АЛУ (сумма частичных произведений);

R3 – регистр сдвига множителя;

SM – регистр АЛУ суммы множимого и суммы частичных произведений;

RC – регистр для формирования старшей части произведения.

22. АЛУ для выполнения операций деления над числами с фиксированной точкой.

Особенности выполнения операции деления чисел с фиксированной запятой, представленных в прямом коде:

Функционирование АЛУ при выполнении операции деления

Выполнение операции деления в АЛУ сводится к последовательности операций сложения и сдвига. Существующие способы деления можно разделить на две группы:

1. Выполнение операции деления методом с восстановлением остатка.
2. Выполнение операции деления методом без восстановлением остатка.

Алгоритмы деления аналогичны алгоритму деления при ручном счете. Рассмотрим особенности деления на примере деления целых чисел. Пусть $Z=X/Y$, где X – делимое, представляемое обычно двойным словом ($2n-1$ информационных разрядов и 1 знаковый разряд), Y – делитель и Z – частное, представляемое словом, содержащим $n-1$ информационных и 1 знаковый разряд. Пусть для простоты X и Y – целые числа, представленные в прямом коде.

Так как Z – слово, то должно выполняться равенство $|Z| < 2^{n-1}$, т.е. $(|X| - |Y| * 2^{n-1}) < 0$. Если неравенство не выполняется, то возникает переполнение разрядной сетки частного и результат получается неверным. Это условие корректного выполнения деления.

Таким образом, для проверки корректности деления надо сдвинуть делитель на $n-1$ разряд влево и вычесть его из делимого. Если результат окажется отрицательным, то деление возможно. Иначе возникнет переполнение разрядной сетки частного.



Таким образом выполнение операции деления начинается с пробного вычитания делителя, сдвинутого на $n-1$ разряд влево, из делимого. Если результат меньше нуля, то деление возможно. Если результат больше нуля, то деление невозможно. Знак частного формируется в результате анализа знаков операндов. Если операнды имеют одинаковые знаки, то частное получается положительным, если знаки различны - частное будет отрицательным. После определения знака результата знаковые разряды операндов обнуляются и деление производится над положительными числами.

Методы деления:

Моделирование функционирования АЛУ при выполнении операции деления методом с восстановлением остатка над числами, представленными в прямом коде

Метод деления с восстановлением остатка состоит в следующем:

1. Из сдвинутого частичного остатка вычитается делитель. Если результат больше либо равен нулю, очередной разряд частного получает значение 1. Если результат меньше нуля, то разряд частного равен нулю и происходит восстановление промежуточного остатка путем прибавления к текущему остатку делителя.
2. Промежуточный остаток сдвигается влево на один разряд и в него заносится очередной разряд делимого. Цикл деления повторяется.

Рассмотрим пример деления целых чисел, представленных в прямом коде, методом с восстановлением остатка.

$$X=15, Y=-3; Z=X/Y;$$

	Знак	Информационные разряды							
X	0	0	0	0	0	0	1	1	1
Y	1	0	0	1	1				

Формируем знак результата путем сложения по модулю 2 знаков делимого и делителя:
 $ZN=0\oplus 1=1$. Обнуляем знаковый разряд делителя, получаем $Y=00011$.

Дополнительный код делителя: $[Y]_{\text{доп.код}}=11101$

Описание операции	Знак	Информационные разряды	
Делимое	+00	0000	1111
Сдвиг делителя влево на 4 разряда, вычитание делителя	1	1101	
Остаток от деления $< 0 \Rightarrow$ деление возможно	+1	1101	1111
Восстановление остатка: прибавление делителя	0	0011	
Восстановленный остаток	0	0000	1111
Сдвиг влево остатка на 1 разряд	+0	0001	1110
Вычитание делителя	1	1101	
1-ый остаток от деления < 0	+1	1110	1110
Восстановление остатка: прибавление делителя	0	0011	
Восстановленный остаток	0	0001	1110
Сдвиг влево остатка на 1 разряд	+0	0011	1100
Вычитание делителя	1	1101	
2-ой остаток от деления > 0	0	0000	1100
Сдвиг влево остатка на 1 разряд	+0	0001	1000
Вычитание делителя	1	1101	
3-ий остаток от деления < 0	+1	1110	1000
Восстановление остатка: прибавление делителя	0	0011	
Восстановленный остаток	0	0001	1000
Сдвиг влево остатка на 1 разряд	+0	0011	0000
Вычитание делителя	1	1101	
4-ый остаток от деления $= 0$	0	0000	0000

Z	1	0	1	0	1
	Знак	1 ост	2 ост	3 ост	4 ост

Без восстановления остатка:

Таким образом для выполнения операции деления методом без восстановления остатка необходимо выполнить следующие действия:

1. Анализ знака промежуточного остатка.
2. Если $A > 0$, то в очередной разряд частного заносят значение 1 и вычитают делитель, если $A < 0$, то в очередной разряд частного заносят значение 0 и прибавляют делитель.
3. Остаток сдвигают влево на один разряд.

Рассмотрим пример деления целых чисел, представленных в прямом коде, методом без восстановления остатка.

$$X=15, Y=-3; Z=X/Y;$$

	Знак	Информационные разряды							
X	0	0	0	0	0	0	1	1	1
Y	1	0	0	1	1				

Формируем знак результата путем сложения по модулю 2 знаков делимого и делителя:

$ZN=0 \oplus 1=1$. Обнуляем знаковый разряд делителя, получаем $Y=00011$.

Дополнительный код делителя: $[Y]_{\text{доп.код}}=11101$.

Описание операции	Знак	Информационные разряды	
Делимое	-00	0000	1111
Сдвиг делителя влево на 4 разряда, вычитание делителя	1	1101	
Остаток от деления $< 0 \Rightarrow$ деление возможно	1	1101	1111
Сдвиг влево остатка на 1 разряд	-1	1011	1110
Сложение с делителем	0	0011	
1-ый остаток от деления < 0	1	1110	1110
Сдвиг влево остатка на 1 разряд	-1	1101	1100
Сложение с делителем	0	0011	
2-ой остаток от деления > 0	0	0000	1100
Сдвиг влево остатка на 1 разряд	-0	0001	1000
Вычитание делителя	1	1101	
3-ий остаток от деления < 0	1	1110	1000
Сдвиг влево остатка на 1 разряд	-1	1101	0000
Сложение с делителем	0	0011	
4-ый остаток от деления $= 0$	0	0000	0000

Z	1	0	1	0	1
	Знак	1 ост	2 ост	3 ост	4 ост

Состав и назначение регистров:

TS – знак операции деления;

SCH – счетчик циклов;

R1 – регистр делителя;

R2, RB – регистр делимого;

R3, SM – регистры АЛУ для организации сдвига;

RC – регистр суммы АЛУ;

RA – регистр для формирования дополнительного и прямого кода делителя;

INP1, INP2 – входные регистры.

23. Особенности выполнения операций в АЛУ над числами с плавающей точкой.

Особенности выполнения операций в АЛУ над числами с плавающей точкой следующие:

1. **Разделение на части:** Числа представлены в форме с мантиссой и порядком, обеспечивая широкий диапазон значений.
2. **Нормализация:** Перед выполнением операций числа обычно нормализуются, чтобы их мантиссы имели одинаковый порядок.
3. **Выравнивание порядка:** Если порядки чисел разные, они выравниваются путем сдвига мантиссы с меньшим порядком.
4. **Выполнение операций:** Проводятся арифметические операции с мантиссами, и при необходимости корректируется порядок результата.

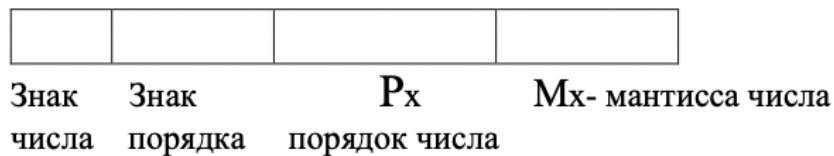
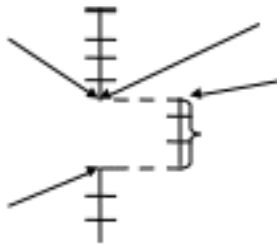


Рис. 2. Формат чисел с плавающей точкой

24. Общие принципы обработки прерываний в ЭВМ.

Выполнение программы



- I. (сохранение основных параметров) запрос на прерывание
- II. (выполнение программы обработки прерывания)
- III. (восстановление основных параметров)

Основные параметры, подлежащие хранению

- Счетчик команд (СЧК),
- Признаковые триггера в АЛУ (флаги),
- Маска.

Дополнительные параметры

- РОНЫ,

- ССП – слово состояния программы (хранит основные параметры),
- Запросы на прерывание, как правило, обрабатываются по окончании выполнения очередной машинной команды.

Для систем **реального времени**, где требуется быстрая реакция на запрос, возможно **прерывание** в процессе выполнения команды.

При возникновении запроса на прерывание по окончании выполнения текущей команды происходят следующие действия:

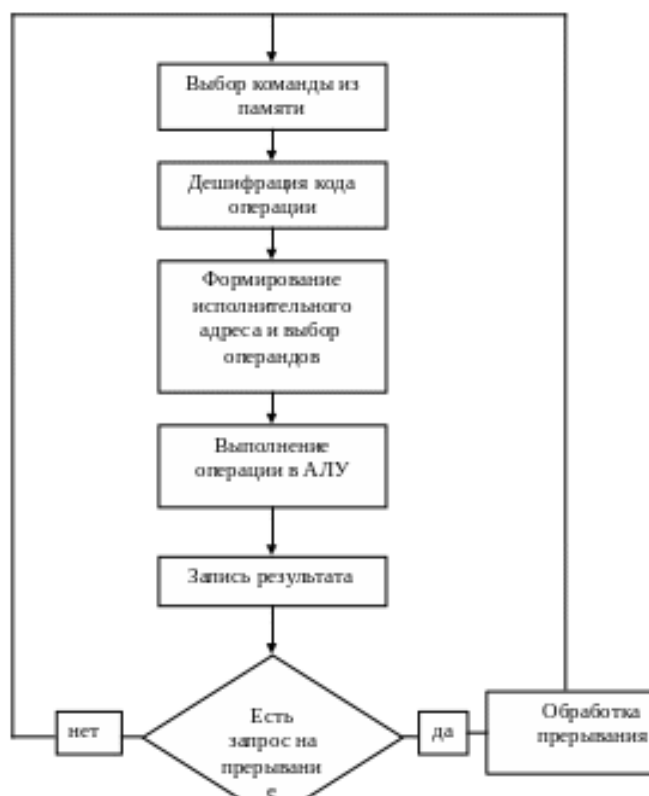
I. Необходимо **сохранить основные параметры** в памяти (счетчик команд, признаковые триггеры АЛУ, маска прерываний).

II. Чтобы выполнить программу обработки прерывания в ЦП, на счетчик команд из некоторой фиксированной ячейки памяти **подается адрес 1-й команды программы обработки прерывания**.

Процесс выполнения программы обработки прерываний условно включает в себя 3 этапа:

1. Сохранение в памяти дополнительных параметров – содержимого РОНов для прерванной программы.
2. Этап непосредственного выполнения программы обработки прерывания.
3. Восстановление из памяти содержимого РОНов для прерванной программы.

III. Чтобы продолжить выполнение прерванной программы, необходимо **восстановить в ЦП содержание регистров ССП**, т.е. основных параметров.

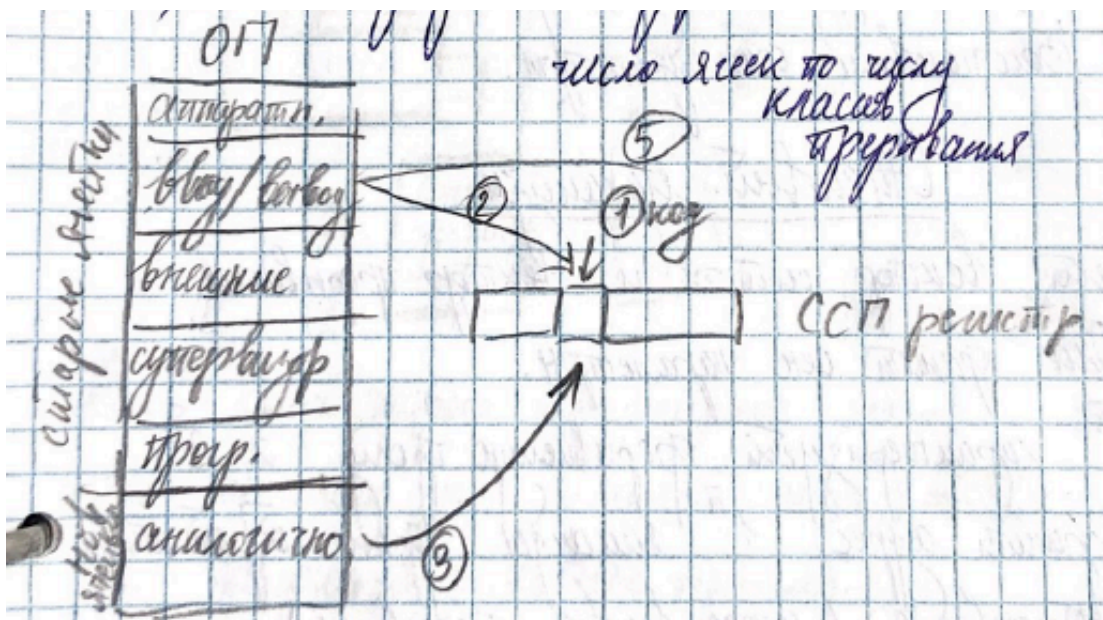


25. Организация системы прерываний с помощью «старых» и «новых» ячеек памяти. (ну вообще нигде, кроме конспекта, нету)

Механизм старых и новых ячеек

Классы прерывания:

1. Аппаратные
2. По вводу-выводу
3. Внешние
4. Супервизор
5. Класс программных требований



1. В ССП заносится код причины прерывания
2. Сохраняем на регистр ССП старую ячейку по фиксированному адресу ввода/вывода. От какого класса пришёл — туда и пишем.
3. На ССП подаём адрес команды. Адрес подаётся из новой ячейки для данного класса. Они заносятся на регистр ССП. СК хранит адрес 1-ой команды.
4. Выполнение команды обработки прерывания:
 - Сохранение РОНов
 - Прерывание
 - Восстановление РОНов
5. Берём старую ячейку, переносим её на ССП. Восстанавливаем основные параметры.

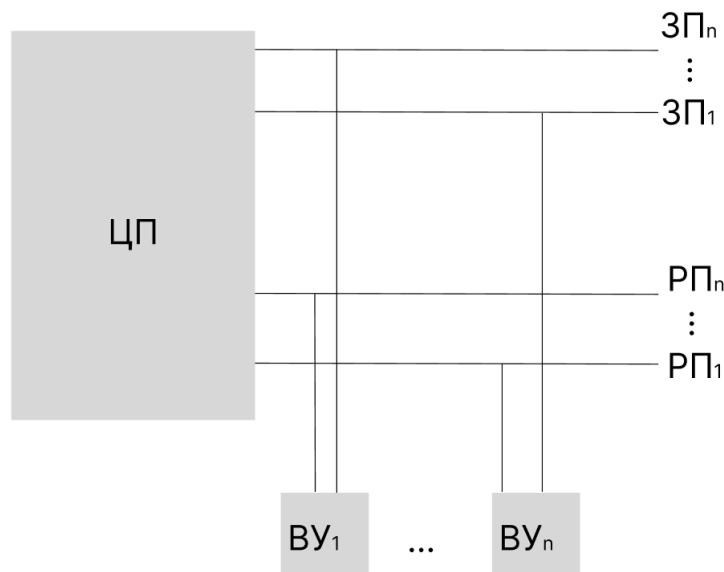
26. Стековый механизм организации прерываний.

Существуют различные ситуации, когда необходимо прервать текущую выполняемую программу и перейти в обработке прерывающей программы (программа обработки прерываний).

Вектор состояния и вектор прерывания хранят основные параметры.

Вектор прерывания - адрес первой программы, которую надо поместить на СЧК.

- 1) Основные параметры сохраняются в вершине стека
- 2) Для внешних прерываний адрес первой команды даст нам внешнее устройство.



Стековый механизм организации прерываний заключается в следующем:

1. При возникновении запроса на прерывание анализируется приоритет обрабатываемой программы в ЦП (порог прерывания) и приоритет запроса на прерывание. Если запрос на прерывание имеет более высокий приоритет, то инициируется процесс обработки прерывания.
2. Вектор состояния для прерванной программы запоминается в стеке. Из некоторой ячейки определяется адрес первой команды обработки прерываний на счетчик команд, т. е. выбирается вектор прерывания.
3. Из некоторой ячейки выбирается вектор прерывания и помещается на регистры ЦП, который содержит адрес 1-й команды программы обработки прерывания.
4. Далее выполняется программа обработки прерывания (3 этапа):
 - сохранение РОНов для прерванной программы;
 - непосредственное выполнение программы обработки прерываний;
 - восстановление сохраненных РОНов для прерванной программы.

5. Из вершины стека выбирается **вектор состояния** для прерванной программы и помещается на **регистры ЦП**.

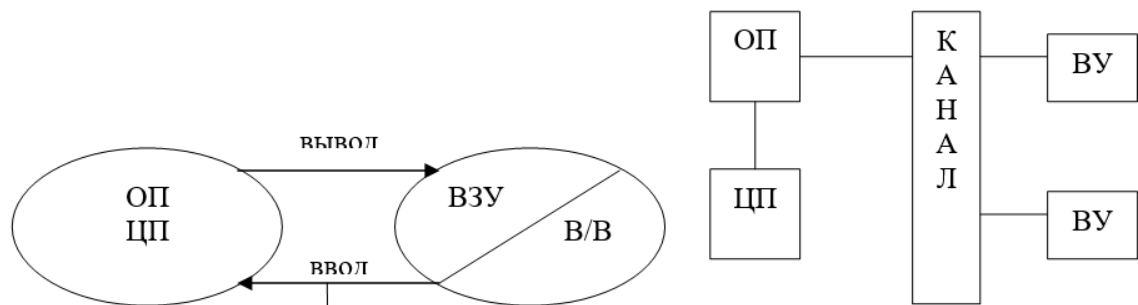
В момент сохранения и восстановления прерывания запрещены.

27. Классификация систем ввода-вывода.

Существуют 3 основных подхода к организации ввода/вывода:

1. Канальный ввод/вывод.
2. Магистральный ввод/вывод.
3. Радиальный ввод/вывод.

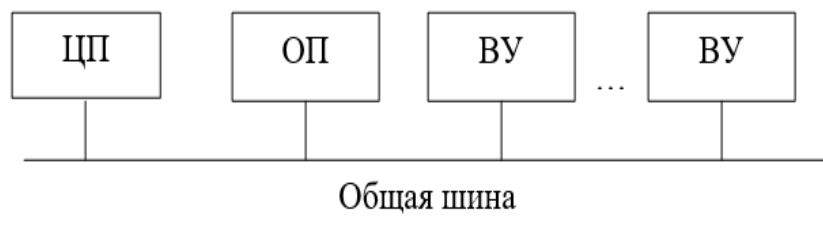
Канальный ввод/вывод



Каналы представляют собой **специализированные процессоры**, предназначенные специально для организации в/в, которые выполняют канальную программу.

ЦП инициирует в/в, а далее продолжает выполнять программу ЦП. Канал, восприняв запрос по в/в, функционирует параллельно с ЦП и занимается в/в информации. Таким образом, для этого класса систем **обработка программ ЦП и организация в/в** ведется параллельно.

Магистральная организация ввода-вывода



Через общую шину могут обслуживаться одновременно только 2 устройства. Поэтому, в отличие от предыдущего случая, **параллельное совмещение по времени выполнения программы в ЦП и ввод/вывод невозможно**.

В каждый момент времени одно устройство является ведущим, а остальные являются ведомыми, за исключением ОП, которая всегда ведомая.

В общей шине выделяют 4 группы шин:

- 1) Шины **данных**: по шинам данных в параллельном коде данные пересылаются между ведущим и ведомым устройствами.
- 2) Шины **управления** предназначены для передачи управляющих сигналов.
- 3) **Адресные** шины: при данном способе ввода/вывода существует единое адресное пространство для всех устройств. Здесь не нужны специальные команды в/в, а достаточно команды move (команда пересылки), а адреса определяют между какими устройствами осуществляется передача.

В этом заключается отличие от радиального в/в.

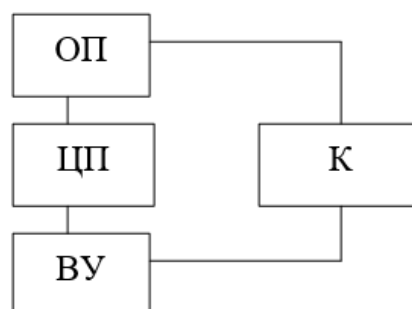
- 4) Шины **арбитража** (шина запроса на прерывание): см. стековый механизм прерывания.

ВУ, готовое к приему или передаче 1-2 байт информации, выставляет сигнал прерывания на шину запроса. Запрос обрабатывается и в качестве программы обработки прерывания будет выступать программа в/в, под управлением которой 1-2 байта информации будут передаваться между двумя устройствами.

Такой программно управляемый в/в характерен для медленных устройств. Для быстродействующих систем используют прямой доступ к памяти.

Быстродействующие устройства подключаются к специальной шине запроса на прерывание, которая имеет наивысший приоритет. При поступлении запроса на шину прямого доступа к памяти общая шина освобождается для в/в. Однако программа, обрабатываемая в ЦП, с обработки не снимается, т.е. стековый механизм реализации прерывания в этом случае не работает.

Радиальная организация ввода-вывода



В отличие от магистрального ввода/вывода имеется отдельное адресное пространство. Это значит, что ВУ может иметь один и тот же адрес, что и ячейки памяти. Используются команды в/в.

28. Ввод-вывод при использовании каналов. Типы каналов.

Каналы и процессоры ввода/вывода

По мере развития систем В/ВЫВ их функции усложняются. Главная цель такого усложнения — максимальное высвобождение ЦП от управления процессами

ввода/вывода. Некоторые пути решения этой задачи уже были рассмотрены. Следующими шагами в преодолении проблемы могут быть:

1. Расширение возможностей МВВ и предоставление ему прав процессора со специализированным набором команд, ориентированных на операции ввода/вывода. ЦП дает указание такому процессору В/ВЫВ выполнить хранящуюся в памяти ВМ программу ввода/вывода. Процессор В/ВЫВ извлекает и исполняет команды этой программы без участия центрального процессора и прерывает ЦП только после завершения всей программы ввода/вывода.
2. Рассмотренному в пункте 1 процессору ввода/вывода придается собственная локальная память, при этом возможно управление множеством устройств В/ВЫВ с минимальным привлечением ЦП.

В первом случае МВВ называют *каналом ввода/вывода* (КВВ), а во втором — *процессором ввода/вывода*. В принципе, различие между каналом и процессором ввода/вывода достаточно условно, поэтому в дальнейшем будем пользоваться термином «канал».

Концепция системы ввода/вывода с КВВ характерна для больших универсальных вычислительных машин (мэйнфреймов), где проблема эффективной организации ввода/вывода и максимального высвобождения центрального процессора в пользу его основной функции стоит наиболее остро. СВВ с каналами ввода/вывода была предложена и реализована в ВМ семейства IBM 360 и получила дальнейшее развитие в семействах IBM 370 и IBM 390.

В ВМ с каналами ввода/вывода центральный процессор практически не участвует в непосредственном управлении периферийными устройствами, делегируя эту задачу специализированному процессору, входящему в состав КВВ. Все функции ЦП сводятся к запуску и остановке операций в КВВ, а также проверке состояния канала и подключенных к нему ПУ. Для этих целей ЦП использует лишь несколько (от 4 до 8) команд ввода/вывода. Например, в IBM 360 таких команд четыре:

- «Начать ввод-вывод»;
- «Остановить ввод-вывод»;
- «Проверить ввод-вывод»;
- «Проверить канал».

КВВ реализует операции ввода/вывода путем выполнения так называемой *канальной программы*. Канальные программы для каждого ПУ, с которым предполагается обмен информацией, описывают нужную последовательность операций ввода-вывода и хранятся в основной памяти ВМ либо локальной памяти канала. Роль команд в канальных программах выполняют *управляющие слова канала (УСК)*, структура которых отличается от структуры обычной машинной команды. Типовое УСК содержит:

- *код операции*, определяющий для КВВ и ПУ тип операции: «Записать» (вывод информации из ОП в ПУ), «Прочитать» (ввод информации из ПУ в ОП), «Управление» (перемещение головок НМД, магнитной ленты и т. п.);
- *указатели* — дополнительные предписания, задающие более сложную последовательность операций В/ВЫВ, например при вводе пропускать отдельные

Структура ВМ с канальной системой ввода/вывода показана на рис. 8.10.

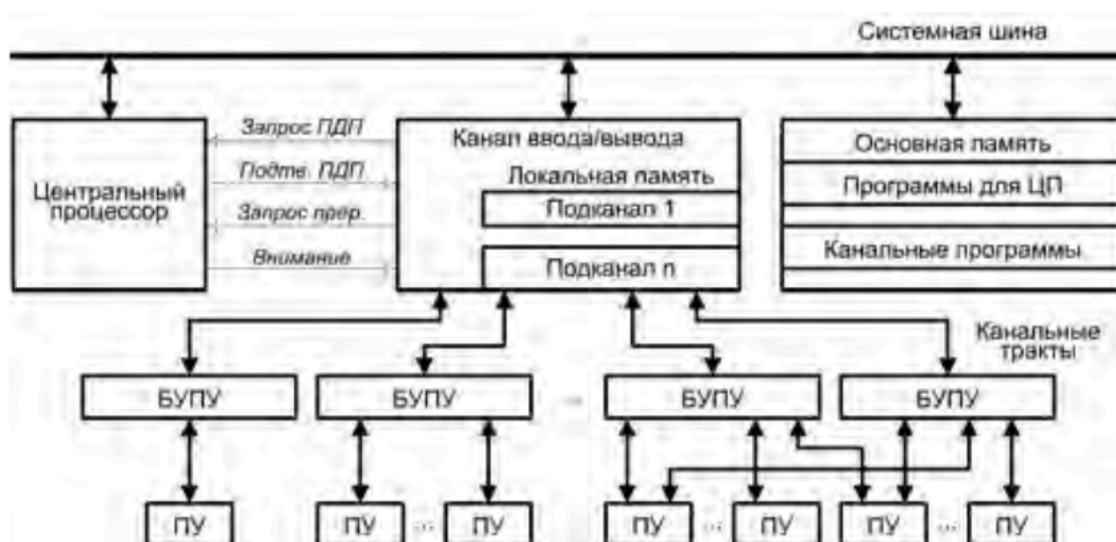


Рис. 8.10. ВМ с канальной системой ввода/вывода

Обмен информацией между КВВ и основной памятью осуществляется посредством системной шины ВМ. ПУ подключаются к каналу не непосредственно, а через блоки управления периферийными устройствами (БУПУ). БУПУ принимает от канала приказы по управлению периферийным устройством (чтение, запись, перемещение носителя или магнитной головки и т. п.) и преобразует их в сигналы управления, свойственные данному типу ПУ. Обычно один БУПУ может обслуживать несколько однотипных ПУ, но для подключения быстродействующих периферийных устройств часто применяются индивидуальные блоки управления. В свою очередь, некоторые ПУ могут подключаться одновременно к нескольким БУПУ. Это позволяет воспользоваться свободным трактом другого БУПУ при занятости данного БУПУ обслуживанием одного из подключенных к нему ПУ. Физически БУПУ может быть самостоятельным устройством или интегрирован с ПУ или каналом.

Обмен информацией между БУПУ и КВВ обеспечивается так называемыми *канальными трактами*. Обычно каждое БУПУ связано с одним из канальных трактов, но возможно также подключение блока управления сразу к нескольким трактам, что дает возможность избежать нежелательных задержек при занятости одного из них.

В пределах канала ввода/вывода считается, что каждое ПУ подключено к своему *подканалу*. Подканалы имеют свои уникальные логические номера, с помощью которых канальная программа адресуется к конкретному ПУ. Физически подканал реализуется в виде участка памяти, в котором хранятся параметры операции ввода/вывода, выполняемой данным ПУ: текущие значения адреса и счетчика данных, код и указатели операции ввода/вывода, адрес следующего УСК и др. Для хранения этих параметров обычно используется локальная память канала.

Обмен информацией между ПУ и ОП, как уже упоминалось, реализуется в режиме прямого доступа к памяти, при этом для взаимодействия ЦП и канала задействованы сигналы Запрос ПДП и Подтверждение ПДП.

Чтобы известить ЦП об окончании текущей канальной программы или об ошибках, возникших при ее выполнении, КВВ выдает в ЦП сигнал Запрос прерывания. В свою очередь, ЦП может привлечь внимание канала сигналом Внимание.

Способ организации взаимодействия ПУ с каналом определяется соотношением быстродействия ОП и ПУ. По этому признаку ПУ образуют две группы: быстродействующие (накопители на магнитных дисках (НМД), накопители на магнитных лентах (НМЛ)) со скоростью приема и выдачи информации около 1 Мбайт/с и медленнодействующие (дисплеи, печатающие устройства и др.) со скоростями порядка 1 Кбайт/с и менее. Быстродействие основной памяти обычно значительно выше. С учетом производительности ПУ в КВВ реализуются два режима работы: *мультиплексный* (режим разделения времени) и *монопольный*.

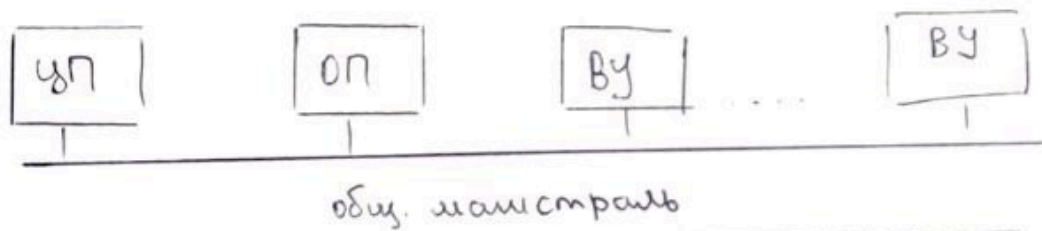
В *мультиплексном режиме* несколько периферийных устройств разделяют канал во времени, при этом каждое из параллельно работающих с каналом ПУ связывается с КВВ на короткие промежутки времени только после того, как ПУ будет готово к приему или выдаче очередной порции информации (байта, группы байтов и т. д.). Такая схема принята в *мультиплексном канале ввода/вывода*. Если в течение

сеанса связи пересылается один байт или несколько байтов, образующих одно машинное слово, канал называется *байт-мультиплексным*. Канал, в котором в пределах сеанса связи пересылка данных выполняется поблочно, носит название *блок-мультиплексного*.

В *монопольном режиме* после установления связи между каналом и ПУ последнее монополизирует канал на все время до завершения инициированной процессором канальной программы и всех предусмотренных этой программой пересылок данных между ПУ и ОП. На все время выполнения канальной программы канал оказывается недоступным для других ПУ. Данную процедуру обеспечивает *селекторный канал ввода/вывода*. Отметим, что в блок-мультиплексном канале в рамках сеанса связи пересылка блока осуществляется в монопольном режиме.

29. Магистральный, радиальный ввод-вывод.

Магистральный ввод / вывод.



Между собой могут работать только два уба.

- либо в/в
- либо обработки данных.

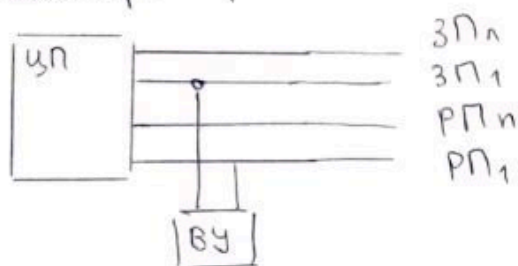
Общая шина вкл. неск. групп шин:

- 1) шина данных
- 2) шин и шин упр. (по шине перед сигналами),
- 3) адресные шин.

Единое адр. пр-во. все регистры ЦП, ВУ, ОП включены в единую нумерацию. Адреса не пересекаются. Не имеют специальные операции В/В

Раздельное адр. пр-во: адр. не уникальны. Здесь исп. спец. код. для в/в.

- 4) Шины арбитража.



В лекциях про радиальный в/в ни слова(

На рис. 13.2 представлены радиальный и магистральный интерфейсы, соединяющие центральный модуль (ЦМ) и другие модули (компоненты) системы ($M_1, \dots, M_n$).

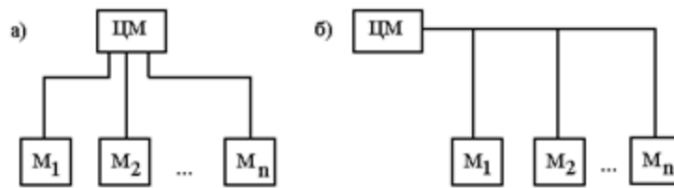
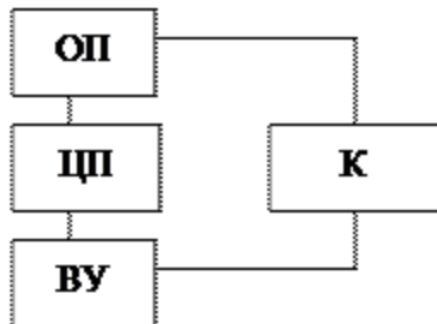


Рис. 13.2. Радиальный (а) и магистральный (б) интерфейсы

Радиальный интерфейс позволяет всем модулям ($M_1, \dots, M_n$) работать независимо, но имеет максимальное количество шин.

Магистральный интерфейс (общая шина) использует принцип разделения времени для связи между ЦМ и другими модулями. Он сравнительно прост в реализации, но лимитирует скорость обмена.

Радиальный ввод/вывод

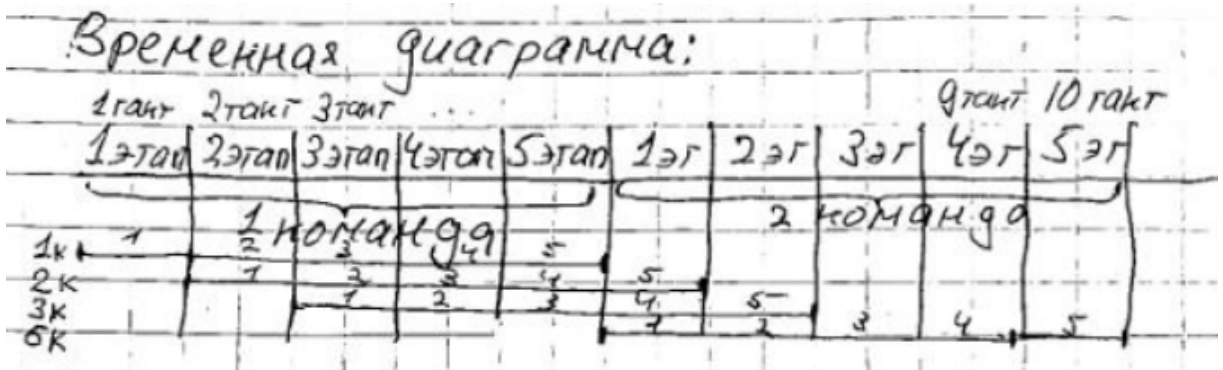


В отличие от магистрального ввода/вывода имеется отдельное адресное пространство. Это значит, что ВУ может иметь один и тот же адрес, что и ячейка памяти, использующая команды в/в.

Т.е. для каждого внешнего устройства используется свой канал связи.

30. Конвейерная обработка команд.

При конвейерной обработке команд они разбиваются на этапы, каждый из которых реализуется на отдельном функциональном блоке. Происходит совмещение обработки различных команд во времени.



В 1-ом такте на 1-ый блок конвейера поступает 1-ая команда. В следующем такте она переходит в следующий блок конвейера, а на 1-ый блок поступает 2-ая команда, и т.д.

При последовательной обработке за 10 тактов было обработано 2 команды, а при конвейере — 6 команд (увеличение производительности ступени конвейера).

За каждый такт выдаются результаты одной команды.

Классический конвейер — до 6 ступеней.

Суперконвейер — 14 ступеней.

Суперскалярная обработка предусматривает исправление нескольких различных функциональных блоков на которых параллельно обрабатываются различные команды.

Информационные конфликты:

1. **Конфликт чтения после записи.** Есть $R1 + R2 \rightarrow R3$. Затем $R3$ используется как операнд: $R4 + R3 \rightarrow R5$.

Пока не сделан 1-ый результат — невозможно выполнить следующую команду. А по временной диаграмме будет конфликт — результата ещё нет, а он должен уже быть записан.

Решение: блокировать команду, пока не будет получен результат (блокировка конвейера, т.е. все зависимые команды и все следующие команды будут задержаны).

2. **Ресурсный конфликт** — наличие свободного блока, который способен обработать команду.

